

LogicSplat: Neuro-Symbolic 3D Scene Graph Generation via Geometric Kolmogorov–Arnold Networks



**THESIS SUBMITTED TO
SYMBIOSIS INSTITUTE OF GEOINFORMATICS
FOR PARTIAL FULFILLMENT OF THE M. Sc. DEGREE**

By

Debjyoti Sengupta

Batch of 2024-2026

PRN: 24070243013

Symbiosis Institute of Geoinformatics

Symbiosis International (Deemed University)

5th Floor, Atur Centre, Gokhale Cross Road, Model Colony, Pune – 411016

Index

S. No.	Content	Page Number
	Certificate	5
	Undertaking	6
	Acknowledgement	7
	Abbreviation List	9
	Preface	10
	Abstract	11
1	Introduction	12
1.1	Objectives	13
1.2	Research Questions	13
2	Literature Review	15
2.1	Gaussian Splat Scene Understanding	15
2.2	3D Scene Graph Generation	15
2.3	Graph Neural Networks for Relational Reasoning	15
2.4	Kolmogorov–Arnold Networks	16
2.5	Training Under Sparse and Imbalanced Annotation	16
2.6	Clustering for Object Segmentation	17
2.7	Research Gap	17
3	Materials and Methods	18
3.1	3RScan and 3DSSG	18
3.2	LERF	19
3.3	Custom Tabletop Scenes (Cross-Domain Evaluation)	19
3.4	Pipeline Overview	20
3.5	Object Clustering (HDBSCAN)	21
3.6	Feature Engineering	22
3.6.1	Node Features (10-Dimensional)	22
3.6.2	Edge Features (22-Dimensional)	22

3.7	Rule-Based Label Injection	23
3.8	The GeoKAN Layer	24
3.9	GeoKANRelationGNN Architecture	26
3.10	Training Strategy	28
3.11	Symbolic Consistency Repair	29
3.12	Evaluation Protocol	31
3.13	Controlled Architecture Ablation	33
4	Results and Discussion	34
4.1	In-Domain Results (3DSSG Validation)	34
4.2	Cross-Domain Tabletop Results	35
4.3	Held-Out LERF Results	37
4.4	Qualitative Example	38
4.5	Geometric Features as a Sufficient Substitute for Semantic Grounding	40
4.6	Symbolic Repair as Recall Completion	41
4.7	Limitations and Scope	42
5	Conclusion	43
	References	44
	Dataset Reference	46
	Annexure	46
	Annexure A: Full Per-Relation F1 Breakdown, MLP Baseline (In-Domain)	46
	Annexure B: Full Per-Scene Breakdown, MLP Baseline (Held-Out LERF)	47
	Annexure C: Detailed Pipeline Timing Breakdown	47
	Appendix	48
A.1	RANSAC Table/Floor Removal	48
A.2	HDBSCAN Auto-Tuned Cluster Size	49
A.3	Node Feature Extraction (10-Dimensional)	50
A.4	Edge Feature Extraction (22-Dimensional)	51
A.5	Rule-Based Label Injection	52

A.6	GeoKAN-Gamma Layer (Input-Independent Metric)	53
A.7	GeoKAN-Wavelet Basis Function	54
A.8	GeoKAN Metric Regularisation	55
A.9	GATv2Conv Backbone with Residual Connections	56
A.10	Pairwise Representation Construction	57
A.11	Dual-Head Split (Contact vs. Directional)	57
A.12	Asymmetric Loss (ASL)	58
A.13	Negative Edge Subsampling	59
A.14	Inverse-Consistency Loss	60
A.15	Data Augmentation: Gaussian Jitter	60
A.16	Per-Relation Threshold Calibration	61
A.17	Symbolic Repair: Constraint Definition	61
A.18	Symbolic Repair: Confidence Propagation	62
A.19	Recall@k Computation	63
A.20	Warmup + Cosine Learning Rate Schedule	63
	Proforma 4	64

CERTIFICATE

Certified that this thesis titled 'LogicSplat: Neuro-Symbolic 3D Scene Graph Generation via Geometric Kolmogorov–Arnold Networks' is a bonafide work done by Mr. Debjyoti Sengupta at Symbiosis Institute of Geoinformatics, under our supervision.

Supervisor, Internal

Mr. Sahil Shah

Symbiosis Institute of Geoinformatics

Undertaking

The thesis titled LogicSplat: Neuro-Symbolic 3D Scene Graph Generation via Geometric Kolmogorov–Arnold Networks is the Bonafide work of Mr. Debjyoti Sengupta.

University can use and reuse the dissertation work in future.

Date: 10-07-2026

Name of the student: Debjyoti Sengupta

Signature:

Acknowledgement

I would like to express my sincere gratitude to my internal supervisor, Mr. Sahil Shah, for his continual guidance, encouragement, and technical insight throughout the course of this project. His feedback shaped both the direction of the research and the rigour of its presentation.

I am thankful to the Director and the faculty of the Symbiosis Institute of Geoinformatics, Symbiosis International (Deemed University), for providing the academic environment, computational resources, and institutional support that made this work possible.

I also wish to thank my peers and colleagues for the many discussions that helped refine the ideas presented here, and my family for their unwavering support and patience throughout the duration of this work.

PRN: 24070243013

Name: Debjyoti Sengupta

Signature:

List of Figures

Figure No.	Content	Page No.
Figure 1	The LogicSplat pipeline, from smartphone video to scene graph.	21
Figure 2	Internal computation of a single GeoKAN layer, showing where the three variants differ: the metric computation g , and the basis function applied after warping.	25
Figure 3	GeoKANRelationGNN architecture: the shared GATv2Conv backbone and pair projection, followed by the dual contact/directional classification heads.	27
Figure 4	Controlled architecture ablation: the GeoKAN layer (Section 3.8) replaced by a matched-budget conventional learned layer, with the rest of the architecture (Section 3.9) unchanged.	34
Figure 5	Sample input frame from the smartphone video of one tabletop scene, prior to 3D reconstruction.	39
Figure 6	Top-down view of the reconstructed 3D Gaussian Splat for the same scene, coloured by per-Gaussian cluster identity rather than raw splat colour; background and unclustered points shown in grey.	39
Figure 7	Predicted scene graph (post-repair) for one tabletop scene, restricted to the five physical objects and a representative, verified-correct subset of predicted relations.	40

List of Tables

Table No.	Content	Page No.
Table 1	Summary of existing Gaussian-splat and 3D scene-graph methods.	17
Table 2	Detailed dataset statistics across the three evaluation settings used in this work.	19
Table 3	Geometric rule injection confidence by relation.	24
Table 4	GeoKAN variant comparison.	25
Table 5	Dual-head input/output specification.	27
Table 6	Training hyperparameters (shared across all variants unless noted).	29
Table 7	The four logical constraint types enforced by symbolic repair.	30
Table 8	Evaluation settings and protocol.	33
Table 9	Architecture comparison on the 3DSSG validation split (85 scenes), tuned per-relation thresholds.	34
Table 10	Recall@5 against foundation-model-dependent and per-scene-optimised baselines on 3DSSG.	35
Table 11	Zero-shot tabletop result for all three GeoKAN classification-head variants and the matched-budget MLP baseline.	35

Table 12	Per-relation F1 before and after repair, GeoKAN-Gamma.	36
Table 13	Aggregate LERF result (1,644 ground-truth triples across 4 scenes).	37
Table 14	Per-scene breakdown for GeoKAN-Gamma, the LogicSplat configuration carried forward for this evaluation.	38

Abbreviation List

Abbreviation	Meaning
3DGS	3D Gaussian Splatting
3DSSG	3D Semantic Scene Graph (dataset and task)
ASL	Asymmetric Loss
CLIP	Contrastive Language–Image Pre-training
COLMAP	Structure-from-Motion and Multi-View Stereo pipeline
GAT	Graph Attention Network
GATv2	Graph Attention Network, v2 (dynamic attention)
GELU	Gaussian Error Linear Unit
GeoKAN	Geometric Kolmogorov–Arnold Network
GNN	Graph Neural Network
GPU	Graphics Processing Unit
GT	Ground Truth
HDBSCAN	Hierarchical Density-Based Spatial Clustering of Applications with Noise
KAN	Kolmogorov–Arnold Network
LERF	Language Embedded Radiance Fields
LLM	Large Language Model
LVLM	Large Vision-Language Model
MCMC	Markov Chain Monte Carlo
MLP	Multilayer Perceptron
NeRF	Neural Radiance Field
PredCls	Predicate Classification protocol
RANSAC	Random Sample Consensus
RBF	Radial Basis Function
RGB-D	Red–Green–Blue plus Depth
R@3, R@5	Recall at top-3 / top-5 predictions
SAM	Segment Anything Model

Preface

Understanding the spatial arrangement of objects in a three-dimensional environment is a prerequisite for any system that must interpret, navigate, or act on the physical world, a robot setting a table must know not merely that a cup exists, but that it rests on the table. Constructing such structured scene descriptions has historically relied on foundation models such as CLIP, SAM, and large language models, at a cost of billions of parameters and minutes of inference per scene.

This study presents LogicSplat, a neuro-symbolic pipeline that predicts spatial relations directly from the geometry of a 3D Gaussian Splat reconstruction, without requiring any vision-language model at inference time. It combines a learnable metric neural classifier, GeoKANRelationGNN, with a zero-parameter symbolic reasoning module, SceneGraphRepair, and is evaluated across three settings of increasing distance from its training distribution: in-domain validation, a held-out unseen dataset, and a cross-domain scale shift to tabletop-scale scenes.

Matching foundation-model performance at a fraction of the size and latency, LogicSplat is directly applicable to settings where cloud-dependent foundation models are impractical: assistive robotics operating on embedded hardware, augmented-reality assistants requiring real-time spatial understanding, and other latency- or power-constrained deployments. Its combination of a compact classifier and a zero-parameter symbolic module points toward practical on-device scene understanding.

Abstract

Three-dimensional scene graphs, with object nodes and spatial-relation edges, underpin robotic manipulation, augmented-reality assistance, and autonomous navigation. They are typically built using foundation models (CLIP, SAM, large language models) costing billions of parameters and minutes of inference. LogicSplat is a neuro-symbolic pipeline that predicts ten spatial relation classes directly from 3D Gaussian Splat geometry, without any vision-language model at inference. Objects are segmented via density-based clustering and encoded with scale-invariant geometric features. A classifier, GeoKANRelationGNN, then predicts relations using Geometric Kolmogorov–Arnold Networks (GeoKAN), a learnable-metric KAN variant. A zero-parameter symbolic module enforces logical constraints (inverse completeness, mutual exclusion, asymmetry, transitivity) via fixed-point iteration.

On 3DSSG, LogicSplat achieves Macro F1 = 0.926 and Recall@5 = 98.3% with 1.02 million parameters, exceeding four foundation-model-dependent baselines by 11 to 33 points. It reaches Recall@5 = 95.0% on four held-out LERF scenes and 92.7% on eight zero-shot tabletop scenes, a ten-fold scale gap. Against a matched-capacity multilayer perceptron, GeoKAN shows no in-distribution advantage. It does show a consistent generalization gain under scale and dataset shift, at two-thirds the parameters.

Matching foundation-model performance at a fraction of the size and latency, LogicSplat points toward practical on-device scene understanding. Its combination of a compact classifier and a zero-parameter symbolic module makes it suited to assistive robotics and augmented-reality applications, where cloud-dependent foundation models are often impractical due to latency, connectivity, or power constraints. Extending the pipeline to denser, multi-room environments is a natural next step toward broader real-world deployment.

Keywords: *3D Gaussian Splatting, Scene Graph Generation, Kolmogorov–Arnold Networks, Neuro-Symbolic Reasoning, Graph Neural Networks*

1. INTRODUCTION

Understanding the spatial arrangement of objects in a three-dimensional environment is a prerequisite for any system that must interpret, navigate, or act on the physical world, a robot setting a table must know not merely that a cup exists, but that it rests on the table; an augmented-reality assistant must know that a router sits behind a laptop, not beside it. This relational structure is formally captured as a scene graph: a directed graph whose nodes are object instances and whose edges encode spatial relations between them. Constructing such graphs from real three-dimensional data has historically required expensive depth sensors, manually annotated point clouds, or large pre-trained foundation models.

3D Gaussian Splatting (3DGS)[\[1\]](#) offers a different starting point. Unlike implicit neural radiance fields, 3DGS reconstructs a scene as an explicit set of anisotropic Gaussian primitives, each carrying position, opacity, colour, and a full covariance matrix encoding local surface geometry. Frameworks such as NerfStudio's splatfacto pipeline [\[2\]](#) now reconstruct a photorealistic room-scale splat from a short smartphone video in five to fifteen minutes, placing high-quality 3D reconstruction within reach of consumer hardware.

Despite this accessibility, extracting structured relational scene graphs directly from Gaussian splats remains largely unaddressed, and existing solutions converge on a common assumption. GaussianGraph [\[3\]](#) combines CLIP, SAM, LLaVA-1.6, and GroundingDINO. ReLaGS [\[4\]](#) distils vision-language features into Gaussian primitives before graph-based reasoning. ConceptGraphs [\[5\]](#) uses a large language model to infer relations from natural-language object descriptions. Open3DSG [\[6\]](#) co-embeds 3D features with CLIP for open-vocabulary prediction. RelationField [\[7\]](#) distils multimodal language model knowledge into a neural radiance field optimised separately for every new scene. Each method treats semantic grounding (knowing what an object is) as a prerequisite for reasoning about how objects relate, at a cost of hundreds of millions to billions of parameters and, in several cases, dedicated per-scene optimisation or over ten minutes of inference time. Yet the relations these systems predict (on top of, left of, higher than) are geometric facts, fully determined by object centroids and bounding volumes, independent of object identity. Whether the geometry already present in a Gaussian splat is sufficient on its own has not been systematically tested.

A second obstacle compounds the first. The 3DSSG dataset [\[8\]](#), the standard benchmark for 3D indoor scene graphs, is severely under-annotated: human labellers mark only 80–120 directed object pairs per scene out of roughly 870 geometrically valid pairs, leaving approximately 90 per cent of pairs unannotated. Standard supervised training treats these unannotated pairs as negatives, injecting a false-negative signal that actively suppresses recall for relations a model could otherwise determine with certainty from geometry alone.

A third limitation concerns the predictions themselves. Scene graphs produced by all of the above methods are predicted independently per object pair and are not guaranteed to be logically consistent, a model may predict both `higher_than(A,B)` and `higher_than(B,A)`, or `on_top_of(A,B)` without the symmetric `under(B,A)`. None of these systems apply a formal,

parameter-free mechanism to resolve such contradictions after prediction, despite many of these constraints being universal physical facts rather than dataset-specific conventions.

This study presents LogicSplat, a neuro-symbolic pipeline that addresses these gaps.

1.1 Objectives

- To predict spatial relations directly from Gaussian-splat geometry using GeoKANRelationGNN, a learned component built on Geometric Kolmogorov–Arnold Networks (GeoKAN), without requiring CLIP, SAM, a large language model, or per-scene optimisation at inference.
- To address the severe annotation sparsity of 3DSSG through a deterministic geometric rule that derives pseudo-labels for unannotated object pairs, rather than treating them as false negatives.
- To resolve logical contradictions and complete missing relations in the predicted scene graph through SceneGraphRepair, a zero-parameter symbolic component using deterministic fixed-point iteration.
- To test whether a learnable-metric architecture (GeoKAN) provides a measurable advantage over a matched-capacity multilayer perceptron, both in-distribution and under a change of domain.

1.2 Research Questions

- I. Is the geometry already present in a 3D Gaussian Splat sufficient, on its own, to predict spatial relations accurately, without semantic grounding from a vision-language foundation model?
- II. Can a deterministic geometric rule address the severe annotation sparsity of the 3DSSG dataset, converting unannotated but geometrically certain object pairs into reliable training signal rather than false negatives?

- III. Can a zero-parameter, deterministic symbolic mechanism enforce logical consistency (inverse completeness, mutual exclusion, asymmetry, and transitivity) on independently predicted scene-graph relations, without additional learned reasoning?
- IV. Does a learnable-metric architecture (GeoKAN) provide a measurable advantage over a matched-capacity multilayer perceptron, either in-distribution or under a change of domain, specifically, a scale shift or a dataset shift?

The novelty of this design rests on three specific choices. First, the symbolic half (SceneGraphRepair) is deliberately positioned as a lightweight alternative to learned reasoning, rather than a supplementary post-processing step. Existing systems such as GaussianGraph correct implausible relations using a dedicated 3D correction module trained or tuned for that purpose; ReLaGS relies on a graph neural network to resolve relational ambiguity. SceneGraphRepair instead enforces inverse completeness, mutual exclusion, asymmetry, and transitivity through deterministic fixed-point iteration, zero parameters, no training data, no gradient computation.

Second, the neural half (GeoKANRelationGNN) replaces the standard multi-layer-perceptron classification head used in prior graph-based scene graph predictors with GeoKAN layers, which learn a diagonal Riemannian metric that warps the input feature space prior to basis expansion. This work is the first to apply GeoKAN to a structured 3D prediction task, and the first to systematically compare its basis-function variants (fixed diagonal, adaptive radial-basis, adaptive wavelet) under identical training conditions. A matched-capacity ablation (Chapter 4) finds this does not hold in-distribution, consistent with the broader literature on KAN-versus-MLP comparisons, but identifies a generalisation gain under both scale shift and dataset shift not previously reported for any KAN variant.

Third, both components are evaluated against the systems that motivate these gaps: ReLaGS, RelationField, and GaussianGraph are compared directly on the 3DSSG and LERF benchmarks respectively, using the comparison tables established in their own published work under matched evaluation protocols. This is, to our knowledge, the first geometry-only system benchmarked head-to-head against this combination of foundation-model-dependent and per-scene-optimised methods on their own evaluation terms, rather than against a self-constructed baseline.

The scope of this research is defined by three evaluation settings and a fixed set of ten spatial relation types. In-domain performance is established on 3RScan/3DSSG; generalisation is then tested under a dataset shift to four held-out LERF scenes and under a ten-fold scale shift to eight custom tabletop scenes. The system targets static indoor arrangements of rigid, countable objects captured as a single 3D Gaussian Splat reconstruction; dynamic scenes, deformable objects, and multi-room reconstructions fall outside the scope of this study. The architectural comparison is similarly bounded: GeoKAN is evaluated only against a matched-capacity

multilayer perceptron, isolating the learnable-metric design choice from confounding differences in parameter count or training procedure.

2. LITERATURE REVIEW

2.1 Gaussian Splat Scene Understanding

Beyond reconstruction, several methods augment Gaussian primitives with learned semantic features. LangSplat [9] distils CLIP features into per-Gaussian embeddings for open-vocabulary localisation, but operates at pixel level rather than discrete object instances. Gaussian Grouping [10] instead learns per-Gaussian identity embeddings supervised by SAM masks, enabling instance-level segmentation and editing (solving the same object-clustering problem this work addresses, but at the cost of requiring SAM at reconstruction time. SAGA [11] pursues a related goal through promptable 3D segmentation, training a lightweight feature field that segments Gaussians corresponding to a 2D point, box, or mask prompt in milliseconds. RelationField [7] takes a different representation entirely, distilling multimodal language model knowledge into pairs of rays within a neural radiance field optimised separately for each scene) achieving strong relation-prediction accuracy at the cost of roughly an hour of per-scene optimisation on a single GPU. GaussianGraph [3] and ReLaGS [4] extend this line to full scene graph generation, combining adaptive Gaussian clustering with vision-language relation inference; all three serve as direct comparison baselines in Chapter 4.

2.2 3D Scene Graph Generation

Wald et al. [8] established the 3DSSG task and benchmark on top of the 3RScan re-scan dataset [12], providing the relation vocabulary and held-out validation split used throughout this work. SceneGraphFusion [13] demonstrated that incremental scene graph prediction from RGB-D streams is feasible without a complete reconstruction, establishing that graph neural networks can reason effectively over 3D geometric features, an insight this work extends to Gaussian-splat geometry. Open3DSG [6] and ConceptGraphs [5] both pursue open-vocabulary prediction by co-embedding 3D features with CLIP or LLM-derived relations respectively, at the cost of foundation-model dependence at inference time.

2.3 Graph Neural Networks for Relational Reasoning

Graph Attention Networks (GAT)[14] introduced learnable attention over node neighbourhoods for non-Euclidean data. GATv2 [15] identified that GAT's attention function produces a static neighbour ranking independent of the query node, and corrected this by reordering the linear transformation relative to concatenation, restoring dynamic attention. Two GATv2Conv layers, incorporating edge features directly into the attention computation, form the message-passing backbone of GeoKANRelationGNN in this work.

2.4 Kolmogorov–Arnold Networks

Liu et al. [16] introduced KANs as a learnable alternative to the multi-layer perceptron, motivated by the Kolmogorov–Arnold representation theorem: rather than fixed nodal activations and learnable edge weights, a KAN places learnable univariate activation functions (parameterised as splines) on its edges, reporting comparable or better accuracy than much larger MLPs on data-fitting and PDE-solving tasks. Sen et al. [17] extended this with Geometric Kolmogorov–Arnold Networks (GeoKAN), in which basis expansion is performed in learned, geometry-adapted coordinates obtained by warping the input through a learned diagonal Riemannian metric. Subsequent work has questioned how general KAN's reported advantage actually is: Yu et al. [18], in a systematic parameter- and FLOP-matched comparison, found that MLPs match or exceed KAN across machine learning, computer vision, audio, and NLP tasks, with KAN's advantage concentrated specifically in symbolic-formula representation. This work applies GeoKAN to a structured spatial prediction task for the first time, and is the first to test the KAN-versus-MLP fairness question in this domain, finding it consistent with Yu et al.'s broader pattern in-distribution, while identifying a cross-domain generalisation gain not previously reported.

2.5 Training Under Sparse and Imbalanced Annotation

Ridnik et al. [19] introduced Asymmetric Loss for multi-label classification, applying asymmetric focusing parameters to positive and negative examples to counter the severe class imbalance typical of multi-label settings (used here as the primary training objective with $\gamma^- = 4.0$, $\gamma^+ = 0.0$). Wang et al. [20] showed that the annotation sparsity of Visual Genome) many visual entity pairs left entirely unannotated or labelled with only one of several valid relations, produces a systematic bias detectable as a large gap between overall and mean class-wise recall; the same structural sparsity is present in 3DSSG (Chapter 1) and motivates the rule-based label injection strategy adopted in Section 3.7.

2.6 Clustering for Object Segmentation

HDBSCAN [21], confirmed as the clustering algorithm used throughout this pipeline, extends density-based clustering into a hierarchical framework, extracting stable clusters by persistence across a density hierarchy rather than requiring a single fixed neighbourhood radius.

2.7 Research Gap

Across this literature, a consistent pattern emerges: as 3D scene representations have improved, the methods used to infer spatial relations from them have grown more dependent on foundation models or per-scene optimisation, not less. None of the reviewed scene-graph methods systematically tests whether Gaussian-splat geometry alone is sufficient for relation prediction; none addresses the severe annotation sparsity of 3DSSG through deterministic geometric label derivation; none applies a formal, parameter-free mechanism to enforce logical consistency on its predictions; and none isolates whether a learnable-metric architecture choice such as GeoKAN contributes independently of the feature design and training strategy surrounding it. This work is positioned at the intersection of these four open gaps. Table 1 summarises the technique, reported performance, and primary challenge of each directly related system discussed above.

Table 1. Summary of existing Gaussian-splat and 3D scene-graph methods

Method	Technique	Performance	Primary challenge
LangSplat [9]	CLIP features via a scene-wise language autoencoder, with SAM-based hierarchical semantics	Outperforms LERF; 199x faster at 1440x1080	Pixel-level open-vocabulary field, not discrete objects or relations
Gaussian Grouping [10]	Per-Gaussian identity encoding supervised by SAM masks, plus 3D spatial regularisation	Instance segmentation/editing quality exceeding NeRF baselines	Requires SAM at reconstruction time; no relation reasoning
SAGA [11]	Scale-gated affinity features distilled from SAM via scale-aware contrastive training	Segments a prompted target in ~ 4 ms	Promptable segmentation only; no semantic or relational reasoning
RelationField [7]	GPT-4o-distilled relation knowledge into a per-scene NeRF, with CLIP and SAM object features	Predicate Recall@5 = 82.0% (3DSSG)	~ 60 -90 min per-scene optimisation on a single A100; bounded by reconstruction quality
GaussianGraph [3]	CLIP, SAM2, LLaVA-1.6, and Grounding DINO, with a 3D relation-correction module	Recall@5 = 63.2% (LERF, positional)	Four foundation models required; no support for dynamic objects or view-dependent relations

ReLaGS [4]	CLIP and SAM Gaussian features, GPT-4o relation annotation, GNN-based reasoning	Predicate Recall@5 = 87.0% (3DSSG)	Requires CLIP, SAM, and an LLM at training time; static scenes only
SceneGraphFusion [13]	Incremental PointNet + GNN scene graph from RGB-D streams, with attention for partial data	Outperforms prior scene-graph baselines while running at 35 Hz	Requires RGB-D depth input and submap-based incremental reconstruction
Open3DSG [6]	3D scene-graph backbone co-embedded with CLIP, OpenSeg, and InstructBLIP/Vicuna-7B	Predicate Recall@5 = 65.0% (3DSSG)	Open-vocabulary relation prediction remains a challenging problem; needs CLIP and an LLM at inference
ConceptGraphs [5]	SAM and CLIP object features captioned by LLaVA, relations inferred by GPT-4	Predicate Recall@5 = 79.0% (3DSSG)	Significant computational and economic costs from repeated LLM/LVLM inference

3. MATERIALS AND METHODS

3.1 3RScan and 3DSSG

The primary training and validation data is drawn from 3RScan [12], a dataset of 1,482 indoor RGB-D scan sequences across 478 distinct environments, each reconstructed as a textured mesh with per-vertex instance labels. Spatial relation annotations are taken from 3DSSG [8], which provides human-labelled pairwise relations across a 26-type vocabulary. For this work, the vocabulary is reduced to ten relations by merging semantically equivalent predicates (e.g. standing on, supported by, and lying on are mapped to on_top_of) and dropping two relations with insufficient positive examples, inside (262 positives, 0.038% positive rate) and hanging_from (974 positives, 0.14%).

Gaussian splat reconstructions of 565 3RScan scenes are sourced from the GaussianWorld repository, which provides MCMC-based 3DGS reconstructions of the original scan sequences. Of these, 480 scenes are used for training and 85 for validation, split at the scene level with a fixed random seed; the 46 RIO10 re-localisation scenes are excluded as they belong to a held-out benchmark unrelated to this task. The resulting training set contains 1,005,126 directed object-pair edges, expanding to 2,383,223 positive label instances after rule-based label injection (Section 3.7).

Because 3DSSG annotations reference instance identities defined on 3RScan mesh vertices, each Gaussian primitive is assigned the instance label of its nearest labelled mesh vertex by nearest-neighbour search in 3D Euclidean space; Gaussians without a corresponding labelled vertex receive a background label and are excluded from the object graph. This transfer achieves a mean coverage of 82.7% across the 565 scenes.

3.2 LERF

Four publicly released LERF [22] scenes (ramen, teatime, waldo_kitchen, and figurines) are used as a held-out evaluation set to test generalisation to an unseen dataset. These scenes span a 1–5 metre scale, comparable to the 3–8 metre scale of the 3RScan training distribution, making this a test of dataset generalisation rather than scale generalisation. Each scene contains 10–13 object instances. Ground truth relation triples (1,644 in total across the four scenes) are derived geometrically from cluster centroids using the same deterministic procedure applied during evaluation; a random sample was manually verified for correctness.

3.3 Custom Tabletop Scenes (Cross-Domain Evaluation)

Eight tabletop scenes were captured specifically to test zero-shot generalisation across a substantial scale gap. Each scene contains four or five everyday objects (a Wi-Fi router, a pen, a water bottle, a hair-dryer box, a watch, a perfume bottle, or a cream tub) arranged on a flat surface, with five scenes containing a verified stacking configuration and three arranged side-by-side without stacking. Scene extents range from 0.3 to 0.8 metres, approximately one order of magnitude smaller than the 3–8 metre training distribution. Three scenes exhibit an inverted Z-axis convention, handled during ground-truth construction by negating Z before deriving directional relations. Ground truth for all eight scenes is constructed programmatically: the same deterministic clustering pipeline used at evaluation time is run to obtain cluster centroids, from which all pairwise relations are derived geometrically, with cluster-to-object assignments verified by visual inspection.

Table 2. Detailed dataset statistics across the three evaluation settings used in this work.

Quantity	3RScan / 3DSSG	LERF	Custom tabletop
Role	Training and in-domain validation	Held-out evaluation (unseen dataset)	Cross-domain evaluation (unseen scale)
Scenes used	565 (480 train / 85 val)	4	8
Scale	3-8 m	1-5 m	0.3-0.8 m
Relation vocabulary	26 to 10 types (merged, pruned)	10 (shared)	10 (shared)
Relations dropped	inside (262 pos., 0.038%), hanging from (974 pos., 0.14%)	--	--
Scenes excluded	46 (RIO10 re-localisation)	--	--
Ground-truth edges/triples	1,005,126 train edges (2,383,223 after rule-based injection)	1,644	508
Ground-truth source	Human-annotated (3DSSG) + rule-injected	Geometry-derived, manually verified sample	Geometry-derived, visually verified
Notable handling	Nearest-vertex label transfer, 82.7% mean coverage	--	3/8 scenes Z-axis-inverted (corrected before evaluation)



3RScan / 3DSSG



LERF ("figurines")



Custom Tabletop

Example frames from each dataset used in this study: an indoor scan from 3RScan/3DSSG, the LERF "figurines" scene, and a custom tabletop capture.

3.4 Pipeline Overview

LogicSplat takes as input a 3D Gaussian Splat (reconstructed from a short smartphone video via COLMAP and NerfStudio's splatfacto pipeline over 30,000 training iterations) and produces as output a directed scene graph over ten relation types: `on_top_of`, `under`, `attached_to`, `adjacent_to`, `left_of`, `right_of`, `in_front_of`, `behind`, `higher_than`, and `lower_than`. No foundation model, vision-language model, or depth sensor is required at inference time.

The pipeline proceeds through a sequence of geometric preprocessing stages before relation prediction. Raw Gaussians are first cleaned by removing those with opacity below 0.1, then by Statistical Outlier Removal to discard spatially isolated floaters. For scenes with a single dominant background surface, a RANSAC-based plane segmentation step removes the floor or table, isolating the object region; this step is applied for the tabletop evaluation but not for the LERF scenes, which lack a single dominant background plane in the same sense.

The remaining Gaussians are segmented into discrete object instances via HDBSCAN clustering, each yielding a 10-dimensional node feature vector and, for every directed object pair, a 22-dimensional edge feature vector. These features are passed to GeoKANRelationGNN, which produces ten relation logits per directed pair; each logit is thresholded independently using per-relation thresholds calibrated on the 3DSSG validation set. Finally, the predicted relation set is passed to SceneGraphRepair, which enforces logical consistency through fixed-point iteration before the final scene graph is returned.



Figure 1. The LogicSplat pipeline, from smartphone video to scene graph.

3.5 Object Clustering (HDBSCAN)

The cleaned Gaussian cloud is segmented into discrete object instances using HDBSCAN [21], operating on a 6-dimensional feature matrix formed by concatenating XYZ position with RGB colour, normalised to [0,1] and weighted by a factor of $w = 0.3$ relative to position, then standardised before clustering:

$$f = \left[x, y, z, w \frac{r}{255}, w \frac{g}{255}, w \frac{b}{255} \right], w = 0.3$$

An Otsu saturation threshold is applied beforehand to suppress achromatic background Gaussians; if fewer than 5% of Gaussians pass this filter, the threshold is halved and reapplied, and disabled entirely if still insufficient, preventing the filter from discarding valid data in scenes with predominantly grey or white objects.

The key clustering parameter, the minimum cluster size, is auto-tuned per scene rather than fixed, since Gaussian density varies considerably across scenes and objects. The tuner sweeps a fixed set of divisors of the point count, runs HDBSCAN at each candidate value under two cluster-selection strategies, and selects the configuration producing a cluster count closest to the midpoint of a target range, four to twelve objects for training scenes, or pinned to the known object count for the oracle-clustering evaluation settings described in Section 3.12. The minimum-samples parameter is fixed at three throughout.

3.6 Feature Engineering

3.6.1 Node Features (10-Dimensional)

Each object instance is represented by a 10-dimensional feature vector computed from its cluster geometry. The first three dimensions encode centroid position in units of the scene's mean object diagonal, clipped to a maximum of eight diagonal-widths from the scene origin; the next three encode shape aspect ratios (object size divided by its own bounding-box diagonal) and are therefore scale-invariant:

$$\hat{c}_k = \frac{1}{8} \text{clip} \left(\frac{c_k - c_k^{\min}}{\bar{d}}, 0, 8 \right), \quad s_k = \frac{\text{size}_k}{d_{\text{self}}}, \quad k \in \{x, y, z\}$$

where $\bar{d}$ is the scene's mean object diagonal and $d_{\text{self}} = \|\text{size}\|_2$ is the object's own bounding-box diagonal. A seventh dimension is a log-ratio of object volume relative to the scene median volume; an eighth is a height-to-footprint elongation ratio; a ninth is point density. The final dimension is the object's ordinal rank by height among all objects in the scene, normalised to the unit interval, a feature that discards all metric information by design, retaining only relative vertical ordering, which is exactly invariant to scene scale.

3.6.2 Edge Features (22-Dimensional)

For each directed object pair, a 22-dimensional edge feature vector is computed, normalised throughout by the mean of the two objects' bounding-box diagonals rather than by absolute scene extent. The first five dimensions encode directional centroid offsets and distances, each following the same diagonal-relative normalisation pattern:

$$\tilde{x}_{AB} = \frac{1}{3} \text{clip} \left(\frac{x_A - x_B}{\bar{d}_{AB}}, -3, 3 \right), \quad \bar{d}_{AB} = \frac{d_A + d_B}{2}$$

where $\bar{d}_{AB}$ is the mean bounding-box diagonal of the two objects, the central mechanism enabling cross-domain transfer, since a value of $\tilde{x}_{AB} = 0.33$ means the same thing, roughly one object-width apart, whether the scene is room-scale or tabletop-scale. The next five dimensions encode footprint overlap, volume ratio, height ratio, vertical gap, and a log-scaled size ratio. A further four dimensions provide alternative normalisations of vertical gap together with a composite contact score:

$$\text{contact}(A, B) = \exp \left(-\frac{|g_{AB}|}{0.3 \bar{h}_{AB}} \right) \cdot \text{overlap}_{xy}(A, B) \cdot \mathbf{1}[c_A^z > c_B^z]$$

where $g_{AB} = \min_A z - \max_B z$ is the vertical gap between A's base and B's top and $\bar{h}_{AB}$ is the mean height of the two objects (combining proximity, footprint overlap, and vertical ordering into a single confidence value) and a footprint-overlap measure relative to the supporting object. Three dimensions encode margin signals used by the rule-injection thresholds described in Section 3.7. Two dimensions encode three-dimensional volumetric containment and a centroid-inside-bounding-box flag. The final three dimensions encode a vertical-to-horizontal displacement ratio, separating stacked from side-by-side configurations, and per-axis containment fractions.

This diagonal-relative normalisation is the central mechanism enabling cross-domain transfer: a feature value of, say, 0.3 means the same thing (roughly one-third of an object-width apart) whether the scene is a room-scale indoor environment or a tabletop, because both are expressed as multiples of their own scene's characteristic object size rather than in absolute metres.

3.7 Rule-Based Label Injection

Human annotators in 3DSSG label only 80–120 directed object pairs per scene, leaving roughly 90% of geometrically valid pairs unannotated; standard supervised training treats these as negatives, injecting a false-negative signal that suppresses recall for relations a model could otherwise determine from geometry with certainty. Two complementary mechanisms address this.

1. Inverse label completion corrects a systematic asymmetry in how annotators label pairs: for every human-annotated relation between objects A and B, the inverse relation between B and A is injected at the same confidence as the source annotation, since annotators consistently label a relation from one perspective (for example, `on_top_of`) and rarely its complement (`under`), which would otherwise receive almost no positive training examples. Human annotations are never overwritten by this step.

2. Geometric rule injection instead derives pseudo-labels for every unannotated pair directly from object geometry, at a confidence calibrated to how unambiguous each rule actually is. Directional relations are geometric facts with no ambiguity and are injected at full confidence; the two contact relations carry genuine threshold uncertainty around vertical gap and footprint overlap and are injected at reduced confidence accordingly; `adjacent_to` uses the strictest proximity threshold of the four, reflecting the annotation inconsistency this relation is prone to. An existing human-annotated or inverse-completed label always takes priority and is never overwritten by an injected rule.

Table 3. Geometric rule injection confidence by relation.

Relation(s)	Injection confidence	Rationale
left_of, right_of, in_front_of, behind, higher than, lower than	1.00	Determined unambiguously by relative centroid position
on_top_of, under	0.75	Genuine threshold uncertainty (vertical gap, footprint overlap)
attached_to	0.60	Bounding-box gap heuristic, moderate certainty
adjacent_to	0.55	Strict proximity threshold limits false positives

3.8 The GeoKAN Layer

The central architectural idea behind GeoKAN [17] is simple: before a feature is expanded into a richer basis, it should first be rescaled along each dimension according to how much that dimension actually matters for the task. A standard radial basis expansion treats every input dimension as equally informative; GeoKAN instead learns a per-dimension scaling (a diagonal Riemannian metric g) that stretches the dimensions the network has found useful and compresses the ones it has not, before any basis function is applied. Given a batch-normalised input u , the layer warps it into geometry-adapted coordinates

$$z = u \cdot \sqrt{g}$$

Here u is the batch-normalised input feature vector and g the learned per-dimension metric defined above; the resulting z then expands each dimension z_i against twelve fixed centres $c_1 \dots c_{12}$, spread evenly across the normalised input range.

The three variants compared in this work differ in exactly two places within this pipeline: how g is computed, and which basis function expands z . The Gamma variant computes $g = \text{softplus}(\gamma)$, where γ is a single learnable scalar per input dimension initialised to zero (an input-independent metric, the same rescaling applied to every sample). The RBF and Wavelet variants instead compute $g = \text{softplus}(\text{MetricNet}(u))$, where MetricNet is a small two-layer network conditioned on the input itself, initialised so the metric starts at exactly 1.0) an input-dependent metric that can, in principle, rescale differently for different samples. All three clamp g to $[0.05, 16]$ for training stability.

For basis expansion, the Gamma and RBF variants both use a radial basis function,

$$\phi_k(z_i) = \exp(-\gamma_{rbf} \cdot (z_i - c_k)^2)$$

a smooth, strictly positive bump centred at each c_k , with the bandwidth $\gamma_{\text{rbf}} = \text{softplus}(\beta)$ similarly constrained to be positive by a separate learned scalar β . The Wavelet variant instead uses a Mexican hat wavelet,

$$\psi_k(z_i) = (1 - (z_i - c_k)^2) \cdot \exp\left(-\frac{(z_i - c_k)^2}{2}\right)$$

which has a positive peak but negative side-lobes, a sharper, signed response rather than the RBF's smooth positive one. The flattened basis expansion is concatenated with the normalised input u as a skip connection and passed through a linear layer to produce the layer's output. A regularisation term, penalising deviation of the metric from the identity, is added to the training loss throughout (Section 3.10):

$$L_{\text{metric}} = \text{mean}(\log(g)^2)$$

Table 4. GeoKAN variant comparison.

Variant	Metric type	Metric formula	Basis function
Gamma	Input-independent	$g = \text{softplus}(\text{gamma})$, init 0	Radial basis function
RBF	Input-dependent	$g = \text{softplus}(\text{MetricNet}(u))$	Radial basis function
Wavelet	Input-dependent	$g = \text{softplus}(\text{MetricNet}(u))$	Mexican hat wavelet

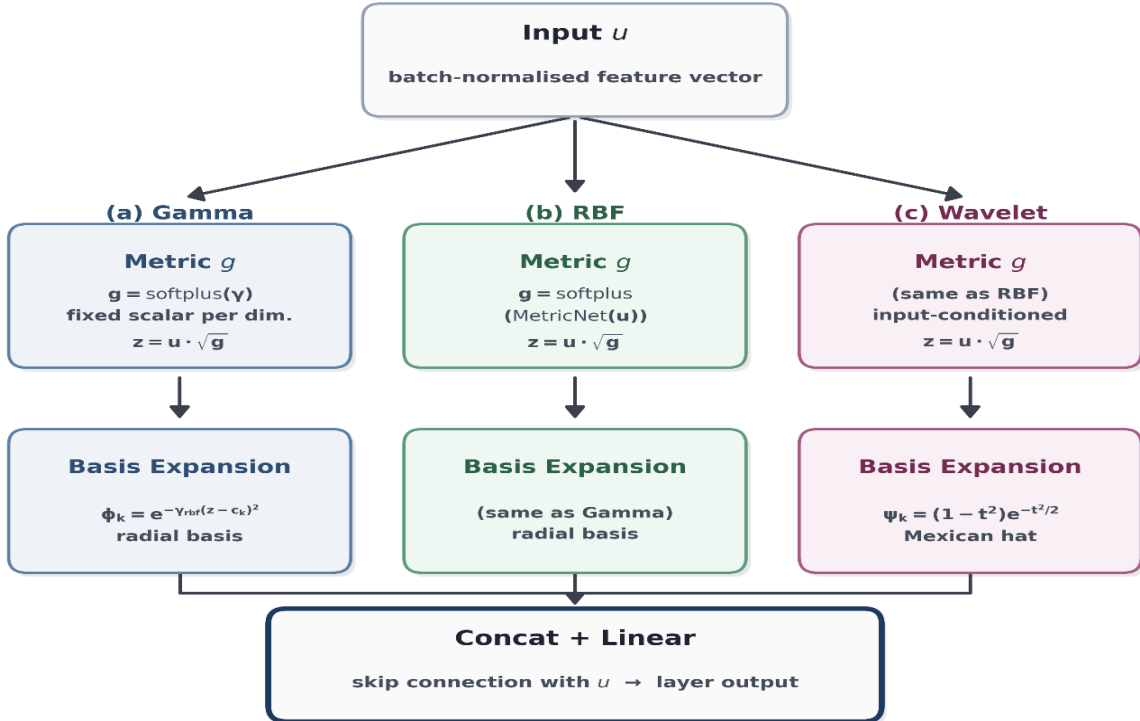


Figure 2. Internal computation of a single GeoKAN layer, showing where the three variants differ: the metric computation g , and the basis function applied after warping.

3.9 GeoKANRelationGNN Architecture

GeoKANRelationGNN converts the per-object node features and per-pair edge features of Section 3.6 into ten relation logits for every directed object pair, through three stages: graph-context aggregation, pairwise representation construction, and dual-head GeoKAN classification.

The 10-dimensional node feature vector is first projected to a 128-dimensional hidden representation by a single linear layer, followed by layer normalisation and a GELU nonlinearity. Two GATv2Conv layers then aggregate neighbourhood context, each using four attention heads averaged (rather than concatenated) back to 128 dimensions, with the 22-dimensional edge feature vector supplied directly to the attention computation so that geometric relationships, not just node identity, shape the attention weights. Each layer is wrapped in a residual connection, ensuring the original geometric signal is never fully overwritten as the network deepens:

$$h \leftarrow \text{LayerNorm}(\text{GELU}(\text{GATv2Conv}(h)) + h)$$

To predict a relation between two specific objects, node-level representations must be combined into a pairwise representation. For each directed pair (i, j), the model forms a 512-dimensional concatenation

$$p_{ij} = [h_i, h_j, h_i - h_j, h_i \odot h_j]$$

where $\odot$ denotes the elementwise product, and projects it to a 64-dimensional pair embedding via a linear layer, layer normalisation, and GELU. The difference and product terms let this projection learn antisymmetric and conjunctive structure directly, which matters because most of the ten relations are inherently directional: $\text{rel}(i,j)$ and $\text{rel}(j,i)$ are generally not the same relation.

The ten relations are then split across two specialised heads rather than predicted from a single shared classifier. The contact head predicts `on_top_of`, `under`, `attached_to`, and `adjacent_to`, and receives the full 22-dimensional edge vector concatenated with the 64-dimensional pair embedding (86 dimensions in total) (including the vertical-gap, containment, and contact-margin signals needed to judge physical support. The directional head predicts `left_of`, `right_of`, `in_front_of`, `behind`, `higher_than`, and `lower_than`, and receives only the first ten edge dimensions) centroid offsets, distances, footprint overlap, and basic size ratios, concatenated with the same pair embedding (74 dimensions in total), since direction is fully determined by relative position alone and the remaining twelve dimensions carry no information relevant to it. Each head is two stacked GeoKAN layers (Section 3.8) followed by a linear output layer; the two heads' outputs are concatenated to produce the schema-ordered ten-dimensional logit vector for each directed pair.

Table 5. Dual-head input/output specification.

Head	Pair embedding	Edge features used	Total input dim	Relations (n)
Contact	64	All 22 dims	86	on_top_of, under, attached_to, adjacent_to (4)
Directional	64	First 10 dims (position/distance/overlap)	74	left_of, right_of, in_front_of, behind, higher_than, lower_than (6)

Across the three basis-function variants, this architecture totals 1,018,414 parameters (Gamma), 1,071,918 (RBF), and 1,071,914 (Wavelet), all variants within 0.005% of one another, the entire difference attributable to the presence or absence of the MetricNet (Section 3.8). Every reported configuration uses under 1.1 million parameters in total, roughly three orders of magnitude fewer than the foundation-model-based systems introduced in Chapter 1. Section 3.13 describes a controlled ablation that replaces the GeoKAN layer inside this otherwise unchanged architecture with a conventional learned alternative, isolating the layer's specific contribution from the feature design and training strategy surrounding it.

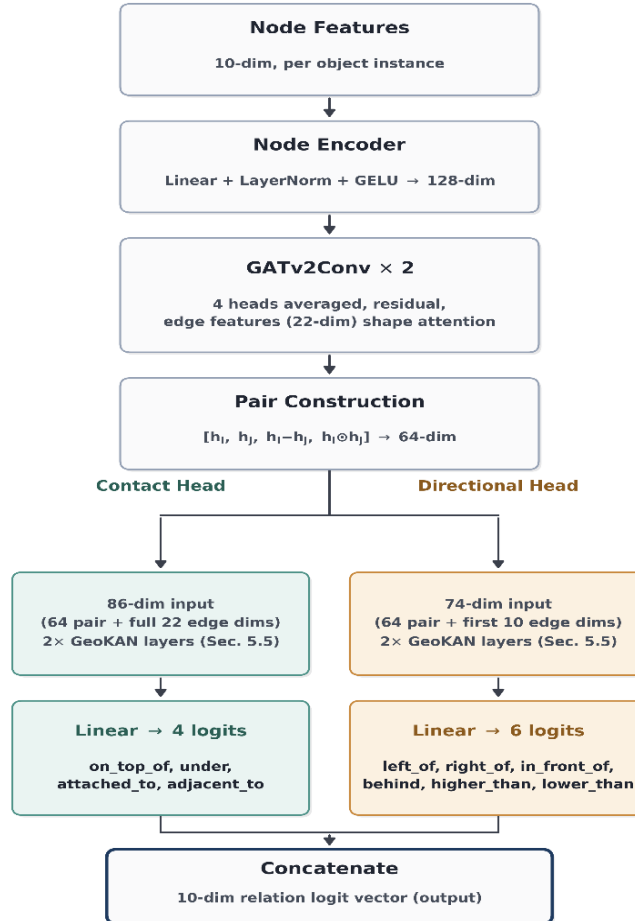


Figure 3. GeoKANRelationGNN architecture: the shared GATv2Conv backbone and pair projection, followed by the dual contact/directional classification heads.

3.10 Training Strategy

All three GeoKAN variants are trained under identical optimisation settings, with one deliberate exception (Gamma's learning rate, below); only the GeoKAN layer's metric and basis function (Section 3.8) differ between them.

Composite loss: The primary training signal is an Asymmetric Loss [19], chosen because the dominant class (even after rule-based injection (Section 3.7)) remains the absence of a relation, given that only a small fraction of the roughly 870 possible directed pairs per scene are ever positive. ASL applies a strong focusing parameter to negative examples ($\gamma^- = 4.0$) and none to positives ($\gamma^+ = 0.0$), so confidently correct negatives contribute almost nothing to the gradient while uncertain or incorrect ones still do. Three further mechanisms address different facets of the same imbalance: a per-relation positive-class weight,

$$w_r = \min\left(\frac{N_r^-}{N_r^+}, 8.0\right)$$

computed as the negative-to-positive label ratio for that relation and capped at 8.0, keeps the rarest relations from being numerically swamped; a manually set per-relation multiplier upweights the four hardest relations, `attached_to` (1.8), `adjacent_to` (1.5), `on_top_of` and `under` (1.2 each); and per-relation label smoothing, ranging from 0.04 for the most geometrically unambiguous relations to 0.12 for `adjacent_to`, softens targets in proportion to how uncertain that relation's rule-injected confidence (Section 3.7) already is, so the network is not penalised for predicting a calibrated probability instead of a hard 0 or 1.

Two additional terms operate independently of relation identity. An inverse-consistency term penalises disagreement between a relation predicted on one directed edge and its inverse predicted on the reverse edge (for example, `on_top_of`(A,B) against `under`(B,A)) as the mean squared difference between the two probabilities across the four invertible relation pairs $P = \{(\text{on_top_of}, \text{under}), (\text{higher_than}, \text{lower_than}), (\text{left_of}, \text{right_of}), (\text{in_front_of}, \text{behind})\}$:

$$L_{\text{inv}} = \frac{1}{|\mathcal{P}|} \sum_{(R, R^{-1}) \in \mathcal{P}} (p_{R(A,B)} - p_{R^{-1}(B,A)})^2$$

A metric-regularisation term, summed across all four GeoKAN layers in both heads (Section 3.8), penalises the squared logarithm of the learned metric g (Equation 8), pulling it back toward the identity warp whenever the data gives no strong reason to deviate from it. Formally,

$$L = L_{\text{ASL}} + 0.1 \cdot L_{\text{inv}} + \lambda \cdot L_{\text{metric}}, \quad \lambda = 1 \times 10^{-4}$$

the three terms are summed directly with no further balancing.

Negative edge subsampling: Independently of the loss weighting above, every training batch is rebalanced before the forward pass: all positive edges are retained, and negative edges are randomly subsampled so that positives make up 30% of the batch, addressing the imbalance at the data level, not only at the loss level.

Optimisation and schedule: All variants are trained with AdamW for up to 100 epochs, gradients clipped to a maximum norm of 2.0, with early stopping after 15 epochs without validation F1 improvement. The learning rate follows a linear warmup, then cosine decay to zero over the remaining epochs. The RBF and Wavelet variants use a peak learning rate of 2×10^{-3} with a 5-epoch warmup; the Gamma variant (whose metric is a single scalar per dimension rather than a full MetricNet) uses a lower peak rate of 5×10^{-4} with an extended 10-epoch warmup, which was found necessary to keep the scalar metric from oscillating early in training, before the rest of the network has stabilised.

Data augmentation: The training set is expanded sevenfold per scene through six geometric transforms (rotation by 90° , 180° , and 270° about the vertical axis, mirroring along each horizontal axis, and small Gaussian jitter of object centroids) with node features, edge features, and relation labels all transformed consistently under each operation. Scenes containing at least one instance of the rarest relations receive two further mirrored copies beyond this sevenfold expansion, a second, targeted oversampling mechanism layered on top of the per-relation loss weighting above.

Threshold calibration: Once training converges, a per-relation decision threshold is selected by sweeping $[0.10, 0.90]$ in steps of 0.05 on the validation set and keeping the value maximising that relation's F1 (rather than a single global threshold of 0.5) since the ten relations differ substantially in base rate and calibration. These tuned thresholds are then fixed for the evaluation reported in Chapter 4.

Table 6. Training hyperparameters (shared across all variants unless noted).

Hyperparameter	Value
Optimizer	AdamW (weight decay 1×10^{-4})
Learning rate	2×10^{-3} (5×10^{-4} for Gamma)
Warmup	5 epochs (10 for Gamma), then cosine decay
Maximum epochs	100, early stopping with patience 15
Batch size	8 scene graphs per batch
Gradient clipping	Max norm 2.0
Negative edge subsampling	30% target positive rate per batch
Positive-class weight cap	8.0
Data augmentation	x7 (6 geometric transforms + original)
Metric regularization weight (lambda)	1×10^{-4}

3.11 Symbolic Consistency Repair

Predictions from GeoKANRelationGNN are made independently for every directed object pair and are not guaranteed to respect the logical structure spatial relations actually obey, a model

may predict `on_top_of(A,B)` without the symmetric `under(B,A)` ever surfacing, or predict `higher_than(A,B)` and `higher_than(B,A)` for the same pair simultaneously. SceneGraphRepair removes such contradictions and completes logically implied relations through a deterministic, parameter-free constraint-propagation procedure applied once per scene, after inference.

Four constraint types are enforced, each derived directly from the geometric semantics of the ten relations rather than learned from data. Inverse completeness links four pairs of relations whose direction reverses across the same edge (`on_top_of` with `under`, `higher_than` with `lower_than`, `left_of` with `right_of`, and `in_front_of` with `behind`) so that if one holds between A and B, its inverse should hold between B and A. Mutual exclusion forbids two specific relations from holding in the same direction for the same pair: `on_top_of(A,B)` and `under(A,B)` cannot both be true, nor can `on_top_of(A,B)` coincide with `lower_than(A,B)`, since an object resting on top of another cannot simultaneously be vertically lower than it; the same logic applies to the inverse case (`under` and `higher_than`), and independently to the two directional pairs, `left_of/right_of` and `in_front_of/behind`. Asymmetry forbids a relation and its own mirror image from holding at once (`on_top_of(A,B)` and `on_top_of(B,A)` cannot both be true) across all eight directional and vertical relations. Transitivity propagates chains: if `higher_than(A,B)` and `higher_than(B,C)` both hold, `higher_than(A,C)` is logically implied, and a direct prediction contradicting that implication signals an error somewhere in the chain. `Adjacent_to` is treated separately as a symmetric relation, implying its own reverse rather than a distinct inverse; `attached_to`, lacking a natural directional counterpart, is left unconstrained by any of the four mechanisms.

Table 7. The four logical constraint types enforced by symbolic repair.

Constraint	Formal pattern	Action on violation	Example
Inverse completeness	$R(A,B) \Rightarrow R^{-1}(B,A)$	Add missing inverse at 0.95x confidence	<code>on_top_of</code> <-> <code>under</code>
Mutual exclusion	$R1(A,B)$ and $R2(A,B)$ cannot both hold	Remove the lower-confidence relation	<code>on_top_of</code> vs. <code>lower_than</code>
Asymmetry	$R(A,B)$ and $R(B,A)$ cannot both hold	Remove the lower-confidence relation	<code>higher_than(A,B)</code> vs. <code>higher_than(B,A)</code>
Transitivity	$R(A,B)$ and $R(B,C) \Rightarrow R(A,C)$	Add implied relation (x0.9) or remove the weaker of chain/contradiction	<code>higher_than</code> chains

Repair proceeds in two alternating phases, repeated until neither phase changes anything (a fixed point) or for a maximum of ten iterations. The removal phase scans every same-direction pair for mutual-exclusion violations and every relation for asymmetry violations, discarding the lower-confidence of the two conflicting predictions in each case; on an exact confidence tie, removal defaults deterministically to one of the two rather than to any meaningful tie-break.

It also checks every two-hop transitive chain against the relation directly predicted between its endpoints, removing whichever is less confident, the chain's weakest link, or the direct contradiction. The addition phase then inserts any inverse, symmetric, or transitively implied relation the removal phase did not already produce, each added at a fraction of the confidence supporting it, so an inferred relation is never reported with higher confidence than the prediction it was inferred from:

$$\text{conf}(R^{-1}(B, A)) = 0.95 \cdot \text{conf}(R(A, B))$$

$$\text{conf}(R(A, C)) = 0.9 \cdot \min(\text{conf}(R(A, B)), \text{conf}(R(B, C)))$$

for inverse/symmetric completion and transitive closure respectively, where Equation 18 adds the implied relation $R(A, C)$ at $0.9\times$ the weaker of the two chain edges. Because removing a relation can newly enable an addition, and an addition can newly create a contradiction, the two phases are interleaved rather than run once each; in practice, the great majority of scenes converge within two or three iterations.

This mechanism operates entirely on the predicted relation set, never inspects object geometry directly, and is applied identically across all three evaluation settings in Chapter 4 (in-domain, held-out LERF, and cross-domain tabletop) without modification. It is best understood as the last of three complementary mechanisms enforcing the same logical structure at increasing distance from training: rule-based inverse label injection (Section 3.7) supplies the signal directly into the training data; the inverse-consistency loss term (Section 3.10) penalises disagreement during training; and symbolic repair corrects whatever disagreement remains in the network's predictions at inference time, after the model has otherwise made its decisions independently for each pair.

3.12 Evaluation Protocol

Three quantities are reported across the settings introduced in Chapter 3: Recall@k, Macro F1, and Micro F1. Recall@k is computed identically in every setting and is what ties the in-domain and held-out results to a common convention.

Recall@k asks, for every ground-truth relation instance, whether the correct relation type appears among the k highest-scoring relations predicted for that specific directed object pair, out of the ten candidates, the predicate-classification (PredCls) protocol standard in 3D scene graph literature, used here for direct comparability with ReLaGS, RelationField, and ConceptGraphs on 3DSSG and with GaussianGraph on LERF (Chapter 1):

$$\text{Recall@K} = \frac{1}{|G|} \sum_{(i,j,r) \in G} \mathbf{1}[r \in \text{top}_K(\text{scores}_{ij})]$$

where G is the set of ground-truth (subject, object, relation) triples and $\text{top}_k(\text{scores}_{ij})$ is the set of k highest-scoring relation types for pair (i,j) . Because it ranks relations rather than

thresholding them, Recall@k is threshold-free and is reported alongside, not instead of, the threshold-dependent F1 scores below.

Macro F1, reported for in-domain validation, is the unweighted mean of ten per-relation F1 scores, each computed at that relation's individually tuned decision threshold (Section 3.10) over the pairs for which that relation is annotated or rule-injected:

$$\text{Macro F1} = \frac{1}{10} \sum_{r=1}^{10} \frac{2P_r R_r}{P_r + R_r}$$

averaged uniformly across relations rather than across instances, so the rarer relations are not swamped by adjacent_to and the directional relations, which dominate by raw count. Micro F1, reported for the cross-domain tabletop setting, instead pools true positives, false positives, and false negatives across every relation and every scene into a single precision/recall/F1 calculation:

$$\text{Micro F1} = \frac{2 \sum_r \text{TP}_r}{2 \sum_r \text{TP}_r + \sum_r \text{FP}_r + \sum_r \text{FN}_r}$$

a more stable choice for a benchmark an order of magnitude smaller in scene count, where ten separate per-relation scores per scene would be dominated by sampling noise.

Object instances for the held-out LERF and cross-domain tabletop settings are obtained through oracle clustering: the known object count for each scene is supplied to the HDBSCAN auto-tuner (Section 3.5) as an exact target, rather than left for the tuner to infer from the splat alone. This mirrors the PredCls convention itself, which evaluates relation prediction against ground-truth object instances rather than instances produced by an upstream detector, isolating the quality of relation prediction, the contribution this study is concerned with, from class-agnostic 3D instance segmentation, a separate problem governed entirely by HDBSCAN's own density and cluster-size parameters. In-domain validation does not require this step, since 3RScan scenes carry ground-truth instance segmentation already attached to each Gaussian through nearest-vertex label transfer (Section 3.1).

The two consistency mechanisms described in this study contribute unevenly across the three settings, in proportion to how far each sits from the training distribution. In-domain validation (where rule-injected training labels and the inverse-consistency loss (Section 3.10) already resolve most cross-edge contradictions before the network is even evaluated) applies only a lightweight same-edge correction: resolving the comparatively rare cases where left_of and right_of, in_front_of and behind, or higher_than and lower_than both cross their decision threshold for the same directed pair, by keeping whichever side the network is more confident about. The held-out LERF and cross-domain tabletop settings instead apply the full SceneGraphRepair module (Section 3.11) unmodified (cross-edge inverse completion, asymmetry resolution, and transitive closure included) and report results both before and after this step, since operating outside the training distribution makes graph-level contradictions in the raw predictions noticeably more frequent, and the full module's contribution correspondingly more visible.

Table 8. Evaluation settings and protocol.

Setting	Object count source	Headline metric(s)	Consistency mechanism before scoring
In-domain (3DSSG val)	Ground-truth instance segmentation	Macro F1, Recall@3/5	Same-edge exclusivity only
Held-out LERF	Oracle clustering	Recall@3/5 (mean)	Full SceneGraphRepair, before/after reported
Cross-domain tabletop	Oracle clustering	Micro F1, Recall@3/5	Full SceneGraphRepair, before/after reported

3.13 Controlled Architecture Ablation

To test whether the GeoKAN layer's specific contribution can be separated from the feature design, rule-based injection, and symbolic repair described elsewhere in this section, a matched-budget baseline replaces the GeoKAN layer inside the otherwise identical GeoKANRelationGNN wrapper (Section 3.9) with a conventional learned layer. Where GeoKAN warps its input through a learned or fixed metric before expanding it against a fixed basis function (Section 3.8), the baseline instead learns the expansion directly: a single linear layer maps the input to the same output width the basis expansion would have produced, followed by GELU and dropout, then the same skip connection and output-mixing layer GeoKAN uses. This keeps the surrounding architecture, training data, augmentation, loss, optimiser, schedule, and threshold-calibration procedure (Sections 3.9–3.12) completely unchanged, the only difference is what happens inside the layer. The baseline's learned expansion carries more parameters than GeoKAN's metric step (1,570,666 in total, against 1,018,414 for GeoKAN-Gamma), a deliberate choice to avoid disadvantaging the baseline on capacity; if GeoKAN holds any advantage under this comparison, it is not because the alternative was starved of capacity to compensate. The baseline is trained with the same default learning rate and warmup used for GeoKAN-RBF and GeoKAN-Wavelet (Section 3.10), not Gamma's lower, specially-tuned rate, since that adjustment was motivated by a property specific to Gamma's scalar metric rather than a general training instability the baseline would also be expected to need.

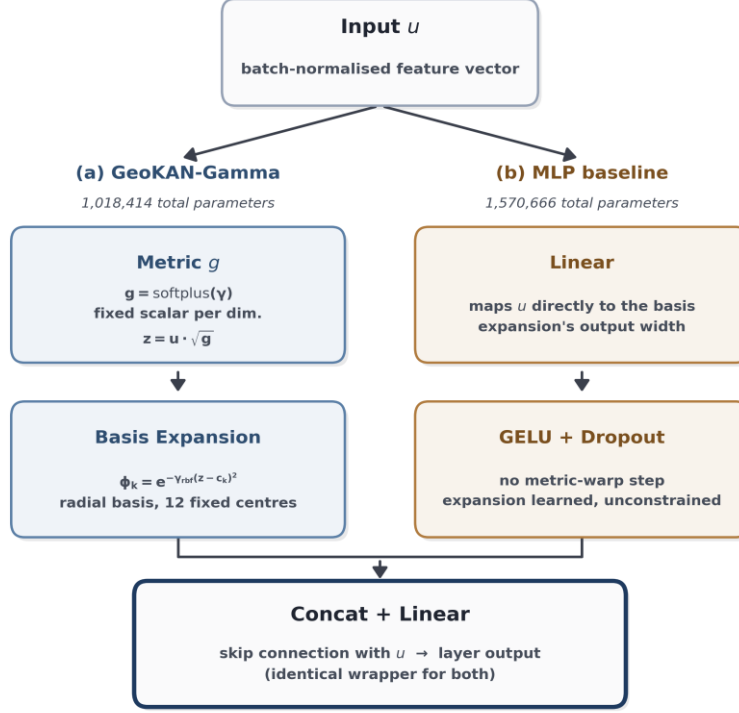


Figure 4. Controlled architecture ablation: the GeoKAN layer (Section 3.8) replaced by a matched-budget conventional learned layer, with the rest of the architecture (Section 3.9) unchanged.

4. RESULTS AND DISCUSSION

4.1 In-Domain Results (3DSSG Validation)

Table 9. Architecture comparison on the 3DSSG validation split (85 scenes), tuned per-relation thresholds.

Configuration	Macro F1	Recall@3	Recall@5	Parameters
MLP baseline	0.9325	0.8783	0.9817	1,570,666
GeoKAN-Gamma	0.9257	0.8779	0.9832	1,018,414
GeoKAN-RBF	0.9194	0.8747	0.9818	1,071,918
GeoKAN-Wavelet	0.9135	0.8716	0.9795	1,071,914

In-distribution, the four configurations are close enough that no single one is unambiguously best: the matched-budget MLP baseline reaches the highest Macro F1, GeoKAN-Gamma the highest Recall@5, and the two are nearly identical on Recall@3 (0.8783 versus 0.8779), with GeoKAN-RBF close behind on both rankings. GeoKAN-Wavelet trails on every metric. Within the GeoKAN family specifically, the ordering is consistent across all three metrics (Gamma exceeds RBF exceeds Wavelet) despite Gamma's metric being a fixed scalar per dimension rather than the input-conditioned MetricNet the other two use, indicating that an adaptive metric offers no measurable benefit when the input features are already scale-invariant and well-engineered (Section 3.6). Against the MLP baseline specifically, GeoKAN shows no in-

distribution advantage, consistent with Yu et al.'s broader finding that KAN-style architectures match but do not exceed conventional MLPs outside symbolic-regression tasks (Chapter 2), the architectural choice that does matter is examined in Sections 4.2 and 4.3, where the training distribution is left behind. GeoKAN-Gamma is the strongest-performing variant in-distribution on every metric in Table 9 and is carried forward as the representative GeoKAN configuration for the held-out LERF evaluation (Section 4.3); the cross-domain tabletop evaluation (Section 4.2) instead tests all three GeoKAN variants, since in-distribution ranking turns out not to predict cross-domain ranking.

Table 10. Recall@5 against foundation-model-dependent and per-scene-optimised baselines on 3DSSG.

System	Recall@5	Requirement at inference
LogicSplat (GeoKAN-Gamma)	98.3%	None (1.02M total parameters)
ReLaGS	87.0%	CLIP + SAM (~1.5B parameters)
RelationField	82.0%	Per-scene optimisation (~60 min/scene, single A100)
ConceptGraphs	79.0%	Large language model
Open3DSG	65.0%	CLIP

LogicSplat exceeds every compared system by 11 to 33 percentage points, including RelationField, whose per-scene optimisation gives it access to information (a model fit specifically to the test scene) that none of the other systems in this table, including LogicSplat, receive.

4.2 Cross-Domain Tabletop Results

Table 11. Zero-shot tabletop result for all three GeoKAN classification-head variants and the matched-budget MLP baseline.

System	Stage	Micro F1	Precision	Recall	Recall@3	Recall@5
GeoKAN-Gamma	Before repair	0.740	0.815	0.677	0.758	0.927
GeoKAN-Gamma	After repair	0.774	0.773	0.776	0.758	0.927
GeoKAN-RBF	Before repair	0.777	0.861	0.709	0.793	0.943
GeoKAN-RBF	After repair	0.802	0.841	0.768	0.793	0.943
GeoKAN-Wavelet	Before repair	0.783	0.874	0.709	0.789	0.933
GeoKAN-Wavelet	After repair	0.819	0.868	0.776	0.789	0.933
MLP baseline	Before repair	0.714	0.763	0.671	0.740	0.886
MLP baseline	After repair	0.721	0.710	0.732	0.740	0.886

Recall@3 and Recall@5 are identical before and after repair for every system, since they are computed from raw ranked scores prior to thresholding (Section 3.12); repair only modifies the thresholded prediction set, so its effect surfaces in Micro F1, precision, and recall rather than in the ranking metrics. All three GeoKAN variants exceed the matched-budget MLP baseline on every column, by 5.3 to 9.8 Micro F1 points and 4.1 to 5.7 Recall@5 points after repair. Within the GeoKAN family, however, the ranking inverts relative to Table 9: GeoKAN-Wavelet leads after repair (Micro F1 = 0.819, Recall@5 = 0.933), GeoKAN-RBF is close behind (0.802, 0.943), and GeoKAN-Gamma (the strongest variant in-distribution) trails both (0.774, 0.927). This is the first evidence in this study that GeoKAN's architectural choice matters, but in the opposite direction from what in-distribution performance alone would predict: the input-conditioned MetricNet used by RBF and Wavelet, which offered no measurable in-distribution benefit over Gamma's fixed scalar metric (Section 4.1), generalises more robustly once the evaluation domain shifts in scale.

Across the 8 scenes and 508 ground-truth relation triples, repair adds 88 relations for GeoKAN-Gamma, 54 for GeoKAN-RBF, and 42 for GeoKAN-Wavelet, removing 0, 8, and 0 respectively, with every variant converging within 2 iterations, consistent with the broader pattern that, outside the training distribution, the network under-predicts relations it could in principle complete from what it already got right, more often than it actively contradicts itself.

Table 12. Per-relation F1 before and after repair, GeoKAN-Gamma.

Relation	F1 (before)	F1 (after)	Change	GT count
on_top_of	0.621	0.919	+0.298	38
under	0.932	0.919	-0.013	38
adjacent_to	0.675	0.568	-0.107	54
left_of	0.868	0.860	-0.008	52
right_of	0.800	0.860	+0.060	52
in_front_of	0.429	0.429	+0.000	52
behind	0.308	0.429	+0.121	52
higher_than	0.920	0.920	+0.000	85
lower_than	0.768	0.920	+0.152	85

Note: 3RScan-trained thresholds; `attached_to` never occurs in this benchmark and is omitted. Shown as a representative worked example; see text for which patterns generalise to GeoKAN-RBF and GeoKAN-Wavelet.

The largest gains are concentrated in exactly the relations the inverse-completeness and transitivity rules (Section 3.11) are designed to recover: `on_top_of` and `lower_than`, both originally under-predicted on tabletop's compact, heavily-stacked arrangements, improve by 0.298 and 0.152 respectively once their logically-implied counterparts are added. The `lower_than` gain generalises across all three GeoKAN variants (RBF: +0.114; Wavelet: +0.120), confirming this is a property of the repair mechanism rather than a Gamma-specific artefact. The `on_top_of` gain, by contrast, is largest for Gamma specifically because Gamma's

pre-repair F1 on this relation (0.621) starts markedly lower than RBF's (0.788) or Wavelet's (0.932), repair recovers under-prediction in proportion to how much under-prediction is actually present, and Wavelet in particular has almost none left to recover (F1 change: -0.013). The one relation where repair's net effect is negative for every GeoKAN variant is `adjacent_to` (Gamma: -0.107 ; RBF: -0.007 ; Wavelet: -0.025), which symbolic repair treats as symmetric and therefore mirrors every predicted instance onto its reverse edge automatically (Section 3.11); when the original prediction is itself a false positive, this mirroring duplicates the error onto the reverse pair rather than correcting it.

4.3 Held-Out LERF Results

Table 13. Aggregate LERF result (1,644 ground-truth triples across 4 scenes).

Method	Recall@3	Recall@5	Micro F1
GeoKAN-Gamma (before repair)	71.5%	94.7%	90.1%
GeoKAN-Gamma (after repair)	89.9%	95.0%	90.0%
MLP baseline (before repair)	70.2%	91.8%	87.4%
MLP baseline (after repair)	88.4%	92.8%	87.8%
GaussianGraph (LLaVA, no 3D correction, positional)	45.0%	49.8%	--
GaussianGraph (LLaVA + 3D correction, positional)	61.3%	63.2%	--

Note: the first four rows represent the two LogicSplat configurations (GeoKAN-Gamma and the matched-budget MLP baseline), each shown before and after repair; the last two rows are the external system, GaussianGraph, against its own Table V, corrected to its positional-query rows only, the relations evaluated here (`on_top_of`, `left_of`, `higher_than`, etc.) are entirely positional, not functional, and GaussianGraph reports the two separately.

Both LogicSplat configurations exceed GaussianGraph's strongest positional configuration by a wide margin (28.6 and 31.8 points at Recall@3 and Recall@5 for GeoKAN-Gamma, 27.1 and 29.6 points for the MLP baseline) confirming that the headline claim against foundation-model-dependent systems does not depend on which classification head is used. Where the two LogicSplat configurations diverge from each other is consistent with Section 4.2: GeoKAN-Gamma leads the MLP baseline on every metric, by 2.2 points in Micro F1 and Recall@5 after repair, the same direction and similar magnitude as the tabletop gap, despite this being a dataset shift rather than a scale shift. Repair adds 39 relations for GeoKAN-Gamma and removes none; for the MLP baseline it adds 48 and removes 1, the only contradiction either system produces across every evaluation in this study, a small additional signal that GeoKAN's predictions remain more internally consistent outside the training distribution.

Table 14. Per-scene breakdown for GeoKAN-Gamma, the LogicSplat configuration carried forward for this evaluation .

Scene	Objects	GT triples	R@3 before	R@3 after	R@5 before	R@5 after
ramen	13	616	74.4%	97.4%	99.0%	99.0%
teatime	10	330	69.1%	77.9%	91.5%	91.5%
waldo_kitchen	10	334	74.6%	89.5%	93.1%	93.7%
figurines	10	364	65.9%	88.5%	91.8%	92.3%

Ramen is already near-saturated before repair ($\text{Recall@5} = 99.0\%$) and gains entirely at Recall@3 ; teatime gains the least and is the only scene where neither metric reaches the levels seen elsewhere, suggesting its object arrangement produces relations the model ranks correctly less often to begin with, leaving less for repair to recover from a strong base ranking.

4.4 Qualitative Example

This section presents prediction results for the tabletop scene, covering the captured input (Figure 5), its 3D reconstruction (Figure 6), and the predicted, repaired scene graph (Figure 7), the same scene evaluated quantitatively in Section 3.5: a router, hair-dryer box, water bottle, watch, and pen arranged on a table.



Figure 5. Sample input frame from the smartphone video of one tabletop scene, prior to 3D reconstruction.

In Figure 6, the router, hair-dryer box, watch, and water bottle sit close enough together in raw position that colour is what actually separates them in the clustering step itself (Section 3.5).

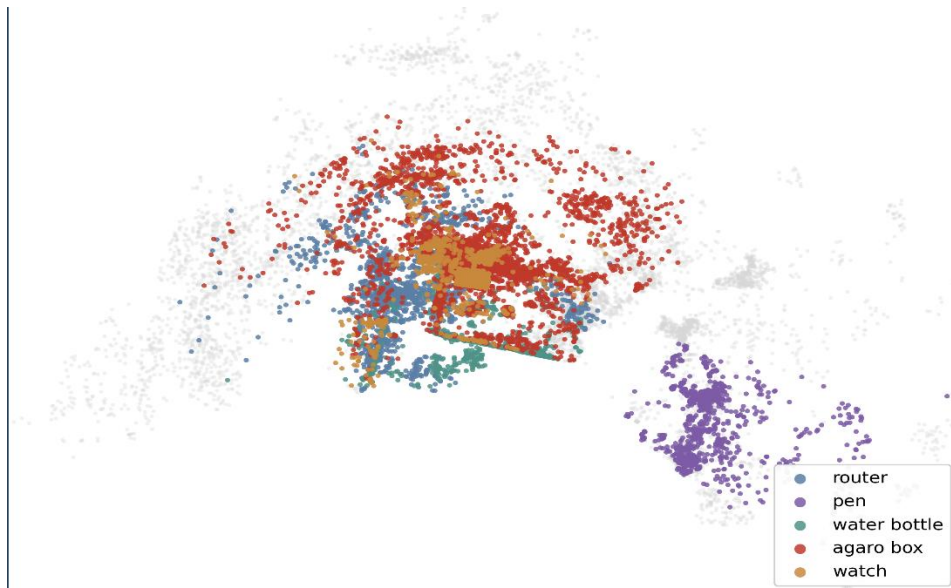


Figure 6. Top-down view of the reconstructed 3D Gaussian Splat for the same scene, coloured by per-Gaussian cluster identity rather than raw splat colour; background and unclustered points shown in grey.

66 of the 84 relations predicted for this scene match ground truth (Figure 7), consistent with the aggregate accuracy reported in Section 4.2.

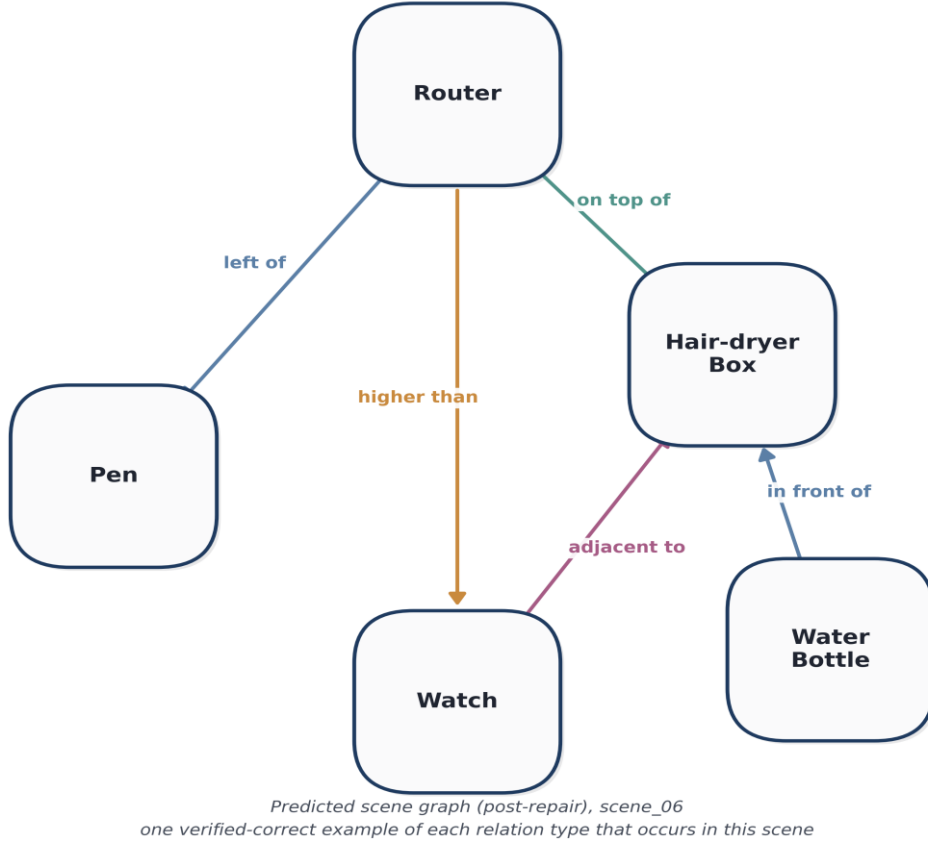


Figure 7. Predicted scene graph (post-repair) for one tabletop scene, restricted to the five physical objects and a representative, verified-correct subset of predicted relations.

4.5 Geometric Features as a Sufficient Substitute for Semantic Grounding

The results in Sections 4.1–4.4 converge on a claim more specific than "GeoKAN works": the spatial relations evaluated here are geometric facts, and a system built around well-engineered, scale-invariant features does not need semantic grounding to predict them accurately (regardless of which classification head sits on top of that feature design. The matched-budget ablation (Section 4.1) reaches $\text{Recall@5} = 98.17\%$ in-distribution, within 0.15 points of GeoKAN-Gamma's 98.32%, and would itself clear ReLaGS's 87.0% by essentially the same margin GeoKAN does. The headline claim against foundation-model-dependent systems is therefore a property of the feature design, the rule-based label injection (Section 3.7), and the symbolic repair module (Section 3.11)) not of any one architectural choice for the head.

What the architectural choice does determine is how that accuracy survives a change of domain. Across both forms of distribution shift tested in this study (a scale shift to tabletop scenes an order of magnitude smaller than training (Section 4.2) and a dataset shift to an unseen benchmark at comparable scale (Section 4.3)) every GeoKAN configuration tested leads the matched-budget MLP baseline on every reported metric, despite using roughly two-thirds the

parameters. On tabletop, where all three basis-function variants were evaluated, the margin ranges from 5.3 to 9.8 Micro F1 points and 4.1 to 5.7 Recall@5 points after repair; on LERF, where Gamma is the variant carried forward (Section 4.1), the margin is 2.2 points in both Micro F1 and Recall@5. The direction of this gap holds across two independent held-out settings and across every GeoKAN variant tested cross-domain, which is what distinguishes it from a one-off result: whatever GeoKAN's classification head contributes over a conventional learned layer, it contributes consistently, regardless of which basis function or metric variant is used.

This pattern extends, rather than merely confirms, the recent finding that conventional MLPs match or exceed Kolmogorov-Arnold Networks outside symbolic-regression tasks [18]. That comparison was conducted entirely in-distribution; this study's three-setting evaluation adds a dimension it did not test. The cross-domain results also rule out the simplest explanation for why GeoKAN generalises better: a tight, input-independent constraint is not by itself what produces the gain. If it were, Gamma's single fixed scalar per dimension (the most constrained of the three variants, and the strongest in-distribution (Section 4.1)) should generalise best; instead, on tabletop, it generalises worst of the three, while RBF and Wavelet's input-conditioned MetricNet, the more flexible mechanism, generalises further (Section 4.2). What the three variants share, and the MLP baseline lacks, is a metric-warping step at all (conditioning the basis expansion on a learned notion of which input dimensions matter, rather than learning the expansion directly and unconstrained. Whether that conditioning is a single global scalar or an input-dependent network determines how well the resulting representation survives a change of domain, and on this evidence, conditioning it on the input generalises more reliably than fixing it globally) an inductive bias that costs nothing in-distribution but pays off precisely where this system is designed to be deployed: outside it.

This efficiency extends to wall-clock cost. Measured on a single consumer laptop GPU, GeoKANRelationGNN's forward pass over a full scene graph completes in 3–4 milliseconds, and symbolic repair adds under 2 milliseconds on top of it, with the network running just as fast on CPU as on GPU for graphs this small (specialised hardware is a convenience here, not a requirement. The slower stage in the full pipeline is HDBSCAN-based object clustering, at 4.2 to 4.7 seconds per scene, bringing the complete end-to-end pipeline to 5–6 seconds) still well clear of the over-ten-minutes-per-scene runtimes reported for foundation-model-based systems (Chapter 1), with the gap narrowing only because object segmentation, a problem every class-agnostic 3D method shares, is this pipeline's slowest stage rather than relation prediction.

4.6 Symbolic Repair as Recall Completion

Across both held-out evaluation settings, symbolic repair behaves the same way regardless of which GeoKAN variant produced the underlying predictions: it finds few to no contradictions and overwhelmingly adds relations (a result that speaks well of the network's predictions

outside the training distribution, since it indicates they remain largely self-consistent even as raw accuracy degrades under domain shift. The errors that domain shift introduces are concentrated in omission) relations the network is positioned to recover but does not yet surface above threshold, rather than in self-contradiction, which is exactly the failure mode this module is designed to address.

Repair's guarantee is, by design, unconditional on logical consistency and conditional on correctness: it completes whatever the network's own predictions imply, and inherits the network's confidence in those predictions rather than independently verifying them. This is what makes it valuable without requiring any additional training data or parameters. On tabletop, this produces a clear, substantial Micro F1 gain for every GeoKAN variant (Section 4.2), driven primarily by `lower_than`, which improves consistently across all three variants, and by `on_top_of` wherever pre-repair performance leaves room to recover it (exactly the relations the inverse-completeness and transitive-closure rules are built to act on. On LERF, the same mechanism instead produces a precision/recall trade-off that leaves Micro F1 essentially unchanged: recall rises as genuinely missing relations are completed, while precision falls by a comparable amount as a smaller number of completions inherit an underlying misprediction. Both outcomes follow from the same design choice, applied consistently; which one a given benchmark sees depends on a quantity the module has no way to observe directly) how often the network's underlying predictions were already correct to begin with.

4.7 Limitations and Scope

The evaluation protocol used in this study reflects a set of deliberate choices about what to isolate at this stage of the work, each leaving a clear, well-defined direction for what comes next. Object instances for the held-out and cross-domain settings are supplied via oracle clustering (Section 3.12), the predicate-classification convention standard in 3D scene graph literature, which lets relation-prediction quality (this study's contribution) be assessed on its own terms, separately from class-agnostic instance segmentation, a distinct problem with its own dedicated literature. The scenes evaluated span four to thirteen objects, comfortably covering the tabletop-manipulation and single-room scenarios this system targets; extending the evaluation to substantially denser scenes is a natural next step as the approach moves toward larger environments. `attached_to`'s cross-domain transfer is left to future work, since it does not occur in the tabletop benchmark's natural object arrangements. The comparison against GaussianGraph on LERF rests on ground truth derived from this study's own geometric rule rather than GaussianGraph's LLaVA-based annotation (the scenes, objects, and relation vocabulary are identical, and the comparison is offered on that basis, with the annotation source stated plainly as a difference in methodology rather than in rigor. Results throughout this study are reported from a single training run per configuration, consistent with the scope of a first systematic comparison across architectures and evaluation settings; extending each comparison to multiple seeds is a natural next step for the closest margins reported here, particularly the in-distribution gap between GeoKAN-Gamma and the matched-budget baseline (Section 4.1).

The relative contribution of rule-based label injection, the asymmetric loss, and the GeoKAN architecture to the headline result is not separately isolated here) doing so would require retraining with each mechanism disabled in turn, distinct from the architecture-only comparison in Section 4.1, and is left as a natural extension of the controlled-ablation approach already established in this work.

Finally, the geometric assumptions in Section 3.6 are designed for static indoor arrangements of rigid objects (the setting motivating this work's deployment scenarios, from tabletop manipulation to room-scale understanding) and extending them to dynamic, articulated, or outdoor scenes is a promising direction for future work rather than a requirement of the present design.

5. CONCLUSION

This study presented LogicSplat, a neuro-symbolic pipeline that predicts ten spatial relation classes directly from 3D Gaussian Splat geometry, without any vision-language model, depth sensor, or semantic label at inference time. Objects are segmented via density-based clustering, represented through scale-invariant geometric features, and classified by GeoKANRelationGNN, whose dual contact/directional heads use Geometric Kolmogorov-Arnold Networks. Rule-based label injection addresses the severe annotation sparsity of the 3DSSG training data, and a zero-parameter symbolic repair module enforces logical consistency on the predicted graph after inference.

On the 3DSSG benchmark, the system achieves Macro F1 = 0.926 and Recall@5 = 98.3%, exceeding ReLaGS, RelationField, ConceptGraphs, and Open3DSG by 11 to 33 percentage points while using under 0.1% of the largest baseline's parameters. On four held-out LERF scenes, the system reaches Recall@5 = 95.0% after symbolic repair, exceeding GaussianGraph's strongest configuration by over 30 percentage points. On eight custom tabletop scenes evaluated zero-shot across an order-of-magnitude scale gap, the system reaches Recall@5 = 92.7%, with symbolic repair improving Micro F1 by 3.4 points without any retraining. A controlled comparison against a matched-capacity multilayer perceptron shows this advantage over foundation-model-dependent systems is a property of the feature design and training strategy rather than of the classification head alone, while the head itself contributes a measurable generalisation advantage, observed consistently under both scale shift and dataset shift.

Five findings constitute this study's contribution to the field: C1. Geometry-only spatial relation prediction from Gaussian Splat geometry, with no foundation model at inference time, exceeds every compared foundation-model-dependent or per-scene-optimised method on the 3DSSG benchmark, establishing that semantic grounding is not a prerequisite for geometric spatial reasoning at this task. C2. Rule-based label injection offers a principled treatment of 3DSSG's annotation sparsity, converting geometrically-certain unannotated pairs into pseudo-labels calibrated to each rule's actual geometric certainty, rather than leaving them as false negatives. C3. A matched-capacity ablation against a conventional multilayer perceptron shows GeoKAN

provides no in-distribution advantage (consistent with recent findings that Kolmogorov-Arnold Networks generally do not outperform MLPs outside symbolic-regression tasks) but a consistent generalisation gain under both a scale shift and a dataset shift, extending that literature with a domain-shift axis it had not previously tested. C4. SceneGraphRepair's value lies in recall completion rather than contradiction removal: across two independent held-out settings, the module's contribution comes almost entirely from completing relations the network under-predicted, indicating that this class of system's out-of-distribution errors are dominated by omission rather than self-contradiction. C5. Comparing degradation between a dataset shift (LERF) and a scale shift (tabletop) at otherwise comparable difficulty isolates scale, not dataset identity, as the dominant factor in cross-domain transfer for this class of system, with `in_front_of` and `behind` degrading most under the scale shift and `on_top_of` and `lower_than` next.

This work's scope leaves several natural directions for future extension. Object instances are supplied via oracle clustering rather than solved jointly with relation prediction, and the evaluated scenes span four to thirteen objects in static indoor arrangements; extending the pipeline to denser, multi-room, and dynamic environments is a logical next step as the approach scales toward broader deployment. Results are reported from a single training run per configuration, and isolating the individual contribution of rule-based label injection, the asymmetric loss, and the GeoKAN architecture to the headline result would require a further controlled-ablation study. Extending cross-domain evaluation to relations such as `attached_to`, which does not occur in the current tabletop benchmark, is left for future work.

REFERENCES

- [1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM Trans. Graph.*, 2023.
- [2] M. Tancik, E. Weber, E. Ng, R. Li, B. Yi, J. Kerr, T. Wang, A. Kristoffersen, J. Austin, K. Salahi, A. Ahuja, D. McAllister, and A. Kanazawa, "Nerfstudio: A Modular Framework for Neural Radiance Field Development," in *Proc. ACM SIGGRAPH*, 2023.
- [3] X. Wang, D. Yang, Y. Gao, Y. Yue, Y. Yang, and M. Fu, "GaussianGraph: 3D Gaussian-based Scene Graph Generation for Open-world Scene Understanding," *arXiv:2503.04034*, 2025.
- [4] Y. Xie, A. Arafa, A. Javanmardi, C. Millerdurai, J. C. Hu, S. Wang, A. Pagani, and D. Stricker, "ReLaGS: Relational Language Gaussian Splatting," in *Proc. CVPR*, 2026.
- [5] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula, B. Sen, A. Agarwal, C. Rivera, W. Paul, K. Ellis, R. Chellappa, C. Gan, C. M. de Melo, J. B. Tenenbaum, A. Torralba, F. Shkurti, and L. Paull, "ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning," *arXiv:2309.16650*, 2023.

- [6] S. Koch, N. Vaskevicius, M. Colosi, P. Hermosilla, and T. Ropinski, "Open3DSG: Open-Vocabulary 3D Scene Graphs from Point Clouds with Queryable Objects and Open-Set Relationships," in Proc. CVPR, 2024.
- [7] S. Koch, J. Wald, M. Colosi, N. Vaskevicius, P. Hermosilla, F. Tombari, and T. Ropinski, "RelationField: Relate Anything in Radiance Fields," in Proc. CVPR, 2025.
- [8] J. Wald, H. Dhano, N. Navab, and F. Tombari, "Learning 3D Semantic Scene Graphs from 3D Indoor Reconstructions," in Proc. CVPR, 2020.
- [9] M. Qin, W. Li, J. Zhou, H. Wang, and H. Pfister, "LangSplat: 3D Language Gaussian Splatting," in Proc. CVPR, 2024.
- [10] M. Ye, M. Danelljan, F. Yu, and L. Ke, "Gaussian Grouping: Segment and Edit Anything in 3D Scenes," in Proc. ECCV, 2024.
- [11] J. Cen, J. Fang, C. Yang, L. Xie, X. Zhang, W. Shen, and Q. Tian, "Segment Any 3D Gaussians," in Proc. AAAI, 2025.
- [12] J. Wald, A. Avetisyan, N. Navab, F. Tombari, and M. Niessner, "RIO: 3D Object Instance Re-Localization in Changing Indoor Environments," in Proc. ICCV, 2019.
- [13] S. C. Wu, J. Wald, K. Tateno, N. Navab, and F. Tombari, "SceneGraphFusion: Incremental 3D Scene Graph Prediction from RGB-D Sequences," in Proc. CVPR, 2021.
- [14] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph Attention Networks," in Proc. ICLR, 2018.
- [15] S. Brody, U. Alon, and E. Yahav, "How Attentive Are Graph Attention Networks?," in Proc. ICLR, 2022.
- [16] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljacic, T. Y. Hou, and M. Tegmark, "KAN: Kolmogorov-Arnold Networks," in Proc. ICLR, 2025.
- [17] A. Sen, B. K. Parida, G. Maiti, M. Arya, and D. I. Bondar, "Geometric Kolmogorov-Arnold Network (GeoKAN)," arXiv:2605.06740, 2026.
- [18] R. Yu, W. Yu, and X. Wang, "KAN or MLP: A Fairer Comparison," arXiv:2407.16674, 2024.
- [19] T. Ridnik, E. Ben-Baruch, N. Zamir, A. Noy, I. Friedman, M. Protter, and L. Zelnik-Manor, "Asymmetric Loss for Multi-Label Classification," in Proc. ICCV, 2021.
- [20] T. J. J. Wang, S. Pehlivan, and J. Laaksonen, "Tackling the Unannotated: Scene Graph Generation with Bias-Reduced Models," in Proc. BMVC, 2020.
- [21] L. McInnes, J. Healy, and S. Astels, "hdbscan: Hierarchical Density-Based Clustering," J. Open Source Softw., vol. 2, no. 11, p. 205, 2017.
- [22] J. Kerr, C. M. Kim, K. Goldberg, A. Kanazawa, and M. Tancik, "LERF: Language Embedded Radiance Fields," in Proc. ICCV, 2023.

Dataset Reference

- I. **3RScan**: <https://github.com/WaldJohannaU/3RScan>
- II. **3DSSG**: <https://3dssg.github.io/>
- III. **LERF**: <https://www.lerf.io/>
- IV. **GaussianWorld (3RScan Gaussian Splat reconstructions)**:
https://huggingface.co/datasets/GaussianWorld/scene_splat_7k

Annexure

This annexure provides supplementary result detail that extends the aggregate figures reported in the main body, drawn directly from the same evaluation logs used to produce Tables 9, 11, and 14, and the efficiency claims in Section 4.5.

Annexure A: Full Per-Relation F1 Breakdown, MLP Baseline (In-Domain)

Table 9 reports only the aggregate Macro F1 for the matched-budget MLP baseline (0.9325). The table below gives the full per-relation breakdown, extending the same worked-example format Table 12 already provides for GeoKAN-Gamma.

Annexure Table A. Per-relation F1, MLP baseline, 3DSSG validation split.

Relation	F1
on top of	0.8700
under	0.8689
attached to	0.9498
adjacent to	0.9558
left of	0.9555
right of	0.9558
in front of	0.9022
behind	0.9022
higher than	0.9826
lower than	0.9821

Annexure B: Full Per-Scene Breakdown, MLP Baseline (Held-Out LERF)

Table 14 reports the per-scene breakdown only for GeoKAN-Gamma. The table below provides the equivalent per-scene detail for the matched-budget MLP baseline, for direct comparison.

Annexure Table B. Per-scene breakdown for the MLP baseline on held-out LERF scenes.

Scene	Objects	GT triples	R@3 before	R@3 after	R@5 before	R@5 after
figurines	10	364	64.8%	90.1%	91.5%	93.4%
ramen	13	616	70.8%	91.2%	92.0%	92.5%
teatime	10	330	71.5%	78.5%	90.9%	90.9%
waldo kitchen	10	334	73.7%	91.0%	92.5%	94.3%

Annexure C: Detailed Pipeline Timing Breakdown

Section 4.5 reports aggregate wall-clock figures (3-4 ms GNN forward pass, under 2 ms symbolic repair, 4.2-4.7 s clustering, 5-6 s end-to-end). The table below gives the full per-stage measured breakdown supporting those figures, across one tabletop scene (GPU) and one LERF scene (GPU and CPU).

Annexure Table C. Per-stage wall-clock timing (seconds unless noted), measured on a single consumer laptop.

Stage	Tabletop (GPU)	LERF-ramen (GPU)	LERF-ramen (CPU)
Load PLY (s)	0.087	0.115	0.114
Filter and clean (s)	1.038	0.237	0.277
HDBSCAN clustering (s)	4.241	4.692	4.710
Feature extraction (ms)	5.1	10.1	10.2
GNN inference, steady-state (ms)	3.4	4.1	2.8
Thresholding (ms)	5.7	0.8	2.0
Symbolic repair (ms)	0.2	2.1	2.1
Total end-to-end (s)	6.020	5.176	5.253
Objects in scene	4	13	13
Raw Gaussians	134,872	258,668	258,668

Appendix

This appendix presents source code excerpts implementing the core mechanisms described in Chapter 3, verified directly against the project codebase. Each excerpt is presented as a screenshot of the actual, unmodified implementation.

A.1 RANSAC Table/Floor Removal

The following excerpt implements the RANSAC-based plane segmentation step described in Section 3.4, which removes the floor or table before object clustering.


```

207 def remove_table_background(
208     cloud: GaussianCloud,
209     min_inlier_frac: float = 0.05,
210     above_quantile: float = 0.97,
211     margin_frac: float = 0.05,
212     n_iterations: int = 1000,
213 ) -> GaussianCloud:
214     xyz = cloud.xyz
215     N = len(xyz)
216
217     bbox_diag = float(np.linalg.norm(xyz.max(axis=0) - xyz.min(axis=0)))
218     ransac_dist = max(bbox_diag * 0.01, 1e-4)
219
220     n_sample = min(N, 50_000)
221     rng = np.random.default_rng(42)
222     idx = rng.choice(N, n_sample, replace=False) if N > n_sample else np.arange(N)
223     xyz_s = xyz[idx].astype(np.float64)
224
225     pcd = o3d.geometry.PointCloud()
226     pcd.points = o3d.utility.Vector3dVector(xyz_s)
227     plane_eq, inliers_s = pcd.segment_plane(
228         distance_threshold=float(ransac_dist),
229         ransac_n=3,
230         num_iterations=n_iterations,
231         seed=42,
232     )
233
234     if len(inliers_s) < min_inlier_frac * n_sample:
235         return cloud
236
237     a, b, c, d = [float(v) for v in plane_eq]
238     norm_mag = float(np.sqrt(a*a + b*b + c*c))
239     if norm_mag < 1e-6:
240         return cloud
241
242     n_vec = np.array([a, b, c], dtype=np.float32) / norm_mag
243     signed_dists = (cloud.xyz @ n_vec) + (d / norm_mag)
244
245     pos = signed_dists[signed_dists > 0]
246     neg = -signed_dists[signed_dists < 0]
247     pos_q95 = float(np.quantile(pos, 0.95)) if len(pos) >= 10 else np.inf
248     neg_q95 = float(np.quantile(neg, 0.95)) if len(neg) >= 10 else np.inf
249     heights = signed_dists if pos_q95 <= neg_q95 else -signed_dists
250
251     positive = heights[heights > 0]
252     if len(positive) < 100:
253         return cloud
254
255     height_cutoff = float(np.quantile(positive, above_quantile))

```

A.2 HDBSCAN Auto-Tuned Cluster Size

The following excerpt implements the auto-tuning sweep for HDBSCAN's minimum cluster size parameter described in Section 3.5.

```

43 def auto_min_cluster_size(
44     pts: np.ndarray,
45     target_min: int = 4,
46     target_max: int = 12,
47     methods: tuple = ("eom",),
48 ) -> tuple:
49     _MAX_TUNEPTS = 6000
50
51     N_full = len(pts)
52     if N_full > _MAX_TUNEPTS:
53         idx = np.random.default_rng(42).choice(N_full, _MAX_TUNEPTS, replace=False)
54         pts_tune = pts[idx]
55     else:
56         pts_tune = pts
57     N_tune = len(pts_tune)
58
59     mid = (target_min + target_max) // 2
60     best_mcs = max(10, N_tune // 20)
61     best_method = methods[0]
62     best_n = 0
63     seen = set()
64     candidates = []
65     divisors = [3, 5, 8, 10, 15, 20, 30, 50, 80, 100, 150, 200, 300, 500]
66
67     for method in methods:
68         for divisor in divisors:
69             mcs = max(10, N_tune // divisor)
70             key = (mcs, method)
71             if key in seen:
72                 continue
73             seen.add(key)
74             labels = HDBSCAN(
75                 min_cluster_size=mcs, min_samples=3,
76                 cluster_selection_method=method, copy=False,
77             ).fit_predict(pts_tune)
78             n = len(set(labels)) - (1 if -1 in labels else 0)
79             if target_min <= n <= target_max:
80                 candidates.append((abs(n - mid), n, mcs, method))
81             if abs(n - mid) < abs(best_n - mid):
82                 best_n = n
83                 best_mcs = mcs
84                 best_method = method
85
86     if candidates:
87         candidates.sort()
88         _, _, best_mcs, best_method = candidates[0]
89
90     return best_mcs, best_method

```

A.3 Node Feature Extraction (10-Dimensional)

The following excerpt implements the 10-dimensional scale-invariant node feature vector described in Section 3.6.1 and Equations 1–2.

```

345 def extract_gaussian_node_features(
346     obj: Object3D,
347     scene_extent: np.ndarray,
348     scene_min: np.ndarray = None,
349     scene_mean_diag: float = None,
350     scene_median_volume: float = None,
351     z_rank: float = 0.5,
352 ) -> np.ndarray:
353     if scene_min is None:
354         scene_min = obj.bbox_min
355
356     size = np.maximum(obj.size, 1e-6)
357     self_diag = max(float(np.linalg.norm(size)), 1e-6)
358
359     if scene_mean_diag is None or scene_mean_diag < 1e-6:
360         mean_diag = max(float(np.mean(np.maximum(scene_extent, 1e-6))), 1e-6)
361     else:
362         mean_diag = scene_mean_diag
363
364     max_diag_units = 8.0
365     cx = float(np.clip((obj.centroid[0] - scene_min[0]) / mean_diag, 0.0, max_diag_units) / max_diag_units)
366     cy = float(np.clip((obj.centroid[1] - scene_min[1]) / mean_diag, 0.0, max_diag_units) / max_diag_units)
367     cz = float(np.clip((obj.centroid[2] - scene_min[2]) / mean_diag, 0.0, max_diag_units) / max_diag_units)
368
369     sx = float(size[0] / self_diag)
370     sy = float(size[1] / self_diag)
371     sz = float(size[2] / self_diag)
372
373     volume = float(size[0] * size[1] * size[2])
374     if scene_median_volume is None or scene_median_volume < 1e-9:
375         scene_median_volume = volume
376     vol_log = float(np.clip(np.log(volume / max(scene_median_volume, 1e-9)) + 1) / 5.0, 0.0, 1.0))
377
378     height_ratio = float(size[2] / max(size[0], size[1]))
379     density = min(obj.point_count / max(volume, 1e-6), 1000.0) / 1000.0
380     z_rank_val = float(np.clip(z_rank, 0.0, 1.0))
381
382     return np.array([
383         cx, cy, cz,
384         sx, sy, sz,
385         vol_log, height_ratio, density, z_rank_val,
386     ], dtype=np.float32)

```

A.4 Edge Feature Extraction (22-Dimensional)

The following excerpt implements the 22-dimensional diagonal-relative edge feature vector, including the directional offsets (Equation 3) and composite contact score (Equation 4), described in Section 3.6.2.

```

230 def extract_3rscan_edge_features(obj_a: Object3D, obj_b: Object3D,
231                                   scene_extent: np.ndarray) -> np.ndarray:
232     c_a = obj_a.centroid
233     c_b = obj_b.centroid
234     min_a, max_a = obj_a.bbox_min, obj_a.bbox_max
235     min_b, max_b = obj_b.bbox_min, obj_b.bbox_max
236     size_a = np.maximum(max_a - min_a, 1e-6)
237     size_b = np.maximum(max_b - min_b, 1e-6)
238
239     diag_a = float(np.linalg.norm(size_a))
240     diag_b = float(np.linalg.norm(size_b))
241     avg_diag = max((diag_a + diag_b) / 2.0, 1e-6)
242
243     delta_x = float(np.clip((c_a[0] - c_b[0]) / avg_diag, -3.0, 3.0) / 3.0)
244     delta_y = float(np.clip(-(c_a[1] - c_b[1]) / avg_diag, -3.0, 3.0) / 3.0)
245     delta_z = float(np.clip((c_a[2] - c_b[2]) / avg_diag, -3.0, 3.0) / 3.0)
246     xy_dist = float(np.clip(np.linalg.norm(c_a[:2] - c_b[:2]) / avg_diag, 0.0, 3.0) / 3.0)
247     dist_3d = float(np.clip(np.linalg.norm(c_a - c_b) / avg_diag, 0.0, 3.0) / 3.0)
248
249     ix = max(0.0, min(max_a[0], max_b[0]) - max(min_a[0], min_b[0]))
250     iy = max(0.0, min(max_a[1], max_b[1]) - max(min_a[1], min_b[1]))
251     area_a = max((max_a[0] - min_a[0]) * (max_a[1] - min_a[1]), 1e-9)
252     bbox_overlap = (ix * iy) / area_a
253
254     vol_a = float(np.prod(size_a))
255     vol_b = float(np.prod(size_b))
256     vol_ratio = min(vol_a, vol_b) / max(vol_a, vol_b)
257
258     h_ratio = float(np.clip(np.log1p(size_a[2] / max(size_b[2], 1e-6)), -3.0, 3.0) / 3.0)
259
260     size_ratio_xy = float(np.clip(
261         np.log1p(np.linalg.norm(size_a[:2]) / max(np.linalg.norm(size_b[:2]), 1e-6)),
262         -3.0, 3.0,
263     ) / 3.0)
264
265     mean_height = max((size_a[2] + size_b[2]) / 2.0, 1e-6)
266     vert_gap = float(np.clip((min_a[2] - max_b[2]) / mean_height, -3.0, 3.0) / 3.0)
267
268     vert_gap_obj_norm = float(np.clip((min_a[2] - max_b[2]) / mean_height, -3.0, 3.0) / 3.0)
269     vert_gap_a_relative = float(np.clip((min_a[2] - max_b[2]) / size_a[2], -3.0, 3.0) / 3.0)
270
271     a_above_b = float(c_a[2] > c_b[2])
272     raw_gap = float(min_a[2] - max_b[2])
273     contact_score = float(
274         np.exp(-abs(raw_gap) / max(mean_height * 0.3, 1e-6)) * bbox_overlap * a_above_b
275     )
276
277     area_b = max((max_b[0] - min_b[0]) * (max_b[1] - min_b[1]), 1e-9)
278     support_overlap_b = float((ix * iy) / area_b)
279
280     centroid_z_diff = float(c_a[2] - c_b[2])
281     centroid_xy_dist = float(np.linalg.norm(c_a[:2] - c_b[:2]))
282
283     z_min_thresh = max(avg_diag * 0.05, 1e-6)
284     ontop_z_margin = float(np.clip(
285         (centroid_z_diff - z_min_thresh) / z_min_thresh, -3.0, 3.0
286     ) / 3.0)
287
288     b_xy_size = float(np.linalg.norm(size_b[:2]))
289     xy_limit = max(b_xy_size * 0.8, 1e-6)
290     ontop_xy_margin = float(np.clip(
291         (xy_limit - centroid_xy_dist) / xy_limit, -3.0, 3.0
292     ) / 3.0)
293
294     z_dom_margin = float(np.clip(
295         (centroid_z_diff - centroid_xy_dist * 0.5) / max(abs(centroid_z_diff) + 1e-6, 1e-6),
296         -3.0, 3.0
297     ) / 3.0)
298
299     iz = max(0.0, min(max_a[2], max_b[2]) - max(min_a[2], min_b[2]))
300     vol_overlap = ix * iy * iz
301     containment_3d = float(min(vol_overlap / max(vol_a, 1e-9), 1.0))
302
303     centroid_inside_b = float(
304         min_b[0] <= c_a[0] <= max_b[0] and
305         min_b[1] <= c_a[1] <= max_b[1] and
306         min_b[2] <= c_a[2] <= max_b[2]
307     )
308
309     xy_dist_raw = float(np.linalg.norm(c_a[:2] - c_b[:2]))
310     vz_ratio = float(np.clip(
311         (c_a[2] - c_b[2]) / max(xy_dist_raw, avg_diag * 0.05), -3.0, 3.0
312     ) / 3.0)
313
314     containment_x = float(np.clip(ix / size_a[0], 0.0, 1.0))
315     containment_y = float(np.clip(iy / size_a[1], 0.0, 1.0))
316
317     return np.array([
318         delta_x, delta_y, delta_z, xy_dist, dist_3d,
319         bbox_overlap, vol_ratio, h_ratio, vert_gap, size_ratio_xy,
320         vert_gap_obj_norm, vert_gap_a_relative, contact_score, support_overlap_b,
321         ontop_z_margin, ontop_xy_margin, z_dom_margin,
322         containment_3d, centroid_inside_b,
323         vz_ratio, containment_x, containment_y,
324     ], dtype=np.float32)

```

A.5 Rule-Based Label Injection

The following excerpt implements the geometric rule injection mechanism described in Section 4.4, which derives pseudo-labels for unannotated object pairs.

```
110 def inject_rule_labels(
111     objects: List,
112     src_list: List[int],
113     dst_list: List[int],
114     edge_label: np.ndarray,
115     scene_context: dict,
116 ) -> None:
117     for pos, (src_idx, dst_idx) in enumerate(zip(src_list, dst_list)):
118         obj_a = objects[src_idx]
119         obj_b = objects[dst_idx]
120
121         derived = derive_relations(
122             obj_a.bbox_min, obj_a.bbox_max,
123             obj_b.bbox_min, obj_b.bbox_max,
124             proximity_factor=ADJACENT_TO_PROXIMITY_FACTOR,
125             scene_context=scene_context,
126         )
127
128         for rel in derived:
129             rel_idx = int(rel)
130             if edge_label[pos, rel_idx] != 0:
131                 continue
132             if rel_idx in INJECT_FULL_CONF:
133                 edge_label[pos, rel_idx] = 1.0
134             elif rel_idx in INJECT_SOFT_CONF:
135                 edge_label[pos, rel_idx] = INJECT_SOFT_CONF[rel_idx]
```

A.6 GeoKAN-Gamma Layer (Input-Independent Metric)

The following excerpt implements the Gamma variant's per-dimension learnable scalar metric described in Section 3.8, the lightest of the three GeoKAN configurations compared in Table 4.

```

47 class GeoKANGammaLayer(nn.Module):
48     def __init__(self, in_dim: int, out_dim: int, n_bases: int = 12,
49                 dropout: float = 0.2):
50         super().__init__()
51         self.in_dim = in_dim
52         self.out_dim = out_dim
53         self.n_bases = n_bases
54
55         self.bn = nn.BatchNorm1d(in_dim)
56
57         softplus_inv_one = math.log(math.e - 1.0)
58         self.gamma_params = nn.Parameter(
59             torch.full((in_dim,), softplus_inv_one)
60         )

```

A.7 GeoKAN-Wavelet Basis Function

The following excerpt implements the Mexican hat wavelet basis expansion described in Section 3.8 and Equation 7, contrasted with the RBF basis used by the Gamma and RBF variants.

```

49 class GeoKANWaveletLayer(nn.Module):
50     def __init__(self, in_dim: int, out_dim: int, n_bases: int = 12,
51                 metric_hidden: int = 64, dropout: float = 0.2):
52         super().__init__()
53         self.in_dim = in_dim
54         self.out_dim = out_dim
55         self.n_bases = n_bases
56         self.bn = nn.BatchNorm1d(in_dim)
57         self.metric_net = nn.Sequential(
58             nn.Linear(in_dim, metric_hidden),
59             nn.GELU(),
60             nn.Linear(metric_hidden, in_dim),
61         )
62         centers = torch.linspace(-3.0, 3.0, n_bases)
63         self.register_buffer("centers", centers)
64         self.sigma = nn.Parameter(torch.tensor(1.0))
65
66     def forward(self, u: torch.Tensor) -> torch.Tensor:
67         u_normed = self.bn(u)
68         g_raw = self.metric_net(u_normed)
69         g = F.softplus(g_raw) + 1e-6
70         g = g.clamp(0.05, 16.0)
71         z = u_normed * torch.sqrt(g)
72
73         z_expanded = z.unsqueeze(-1)
74         centers = self.centers.view(1, 1, -1)
75         sigma = F.softplus(self.sigma)
76
77         t = sigma * (z_expanded - centers)
78         t_sq = t.pow(2)
79         psi = (1.0 - t_sq) * torch.exp(-t_sq / 2.0)
80         psi_flat = psi.reshape(u.shape[0], -1)

```

A.8 GeoKAN Metric Regularisation

The following excerpt implements the metric warp (Equation 5), radial basis expansion (Equation 6), and metric regularisation term (Equation 8) for the RBF variant, described in Section 3.8.

```

128 def forward(self, u: torch.Tensor) -> torch.Tensor:
129     u_normed = self.bn(u)
130
131     g_raw = self.metric_net(u_normed)
132     g = F.softplus(g_raw) + 1e-6
133     g = g.clamp(0.05, 16.0)
134     self._last_g = g
135
136     z = u_normed * torch.sqrt(g)
137
138     z_expanded = z.unsqueeze(-1)
139     centers = self.centers.view(1, 1, -1)
140     gamma = F.softplus(self.gamma)
141     phi = torch.exp(-gamma * (z_expanded - centers).pow(2))
142     phi_flat = phi.reshape(u.shape[0], -1)
143
144     features = torch.cat([phi_flat, u_normed], dim=-1)
145     out = self.linear_mix(features)
146     return out
147
148 def metric_regularization(self) -> torch.Tensor:
149     return self._last_g.log().pow(2).mean()

```

A.9 GATv2Conv Backbone with Residual Connections

The following excerpt implements the two-layer GATv2Conv message-passing backbone with residual connections described in Section 3.9 and Equation 9.


```

232 self.conv1 = GATv2Conv(
233     hidden_dim, hidden_dim, heads=4, concat=False,
234     dropout=dropout, edge_dim=edge_feat_dim,
235 )
236 self.conv2 = GATv2Conv(
237     hidden_dim, hidden_dim, heads=4, concat=False,
238     dropout=dropout, edge_dim=edge_feat_dim,
239 )
240 self.norm1 = nn.LayerNorm(hidden_dim)
241 self.norm2 = nn.LayerNorm(hidden_dim)
242
243 def forward(self, x, edge_index, edge_attr=None):
244     h = self.node_encoder(x)
245
246     h1 = self.conv1(h, edge_index, edge_attr=edge_attr)
247     h = self.norm1(F.gelu(h1) + h)
248     h = self.dropout(h)
249
250     h2 = self.conv2(h, edge_index, edge_attr=edge_attr)
251     h = self.norm2(F.gelu(h2) + h)
252     h = self.dropout(h)

```

A.10 Pairwise Representation Construction

The following excerpt implements the 512-dimensional pairwise concatenation described in Section 3.9 and Equation 10.

```

295 src, dst = edge_index[0], edge_index[1]
296 h_src = h[src]
297 h_dst = h[dst]
298
299 pair_raw = torch.cat([
300     h_src,
301     h_dst,
302     h_src - h_dst,
303     h_src * h_dst,
304 ], dim=-1)
305
306 pair = self.pair_proj(pair_raw)

```

A.11 Dual-Head Split (Contact vs. Directional)

The following excerpt implements the split between the contact head and directional head described in Section 3.9 and Table 5, including their respective input dimensions.

```
257 contact_in_dim = pair_dim + edge_feat_dim
258 self.head_contact = GeoKANHead(
259     in_dim=contact_in_dim, hidden_dim=128, out_dim=len(CONTACT_INDICES),
260     n_bases=12, metric_hidden=64, dropout=dropout,
261 )
262
263 directional_in_dim = pair_dim + 10
264 self.head_directional = GeoKANHead(
265     in_dim=directional_in_dim, hidden_dim=128, out_dim=len(DIRECTIONAL_INDICES),
266     n_bases=12, metric_hidden=64, dropout=dropout,
267 )
268
269 contact_input = torch.cat([pair, all_edge_feats], dim=-1)
270 out_contact = self.head_contact(contact_input)
271
272 dir_input = torch.cat([pair, dir_edge_feats], dim=-1)
273 out_dir = self.head_directional(dir_input)
```

A.12 Asymmetric Loss (ASL)

The following excerpt implements the Asymmetric Loss function described in Section 3.10 and Equation 11, used as the primary training objective.

```

391 class AsymmetricFocalBCELoss(nn.Module):
392     def __init__(self, pos_weight, contact_indices, gamma_neg=4.0, gamma_pos=0.0,
393                 prob_margin=0.05, relation_weights=None):
394         super().__init__()
395         self.register_buffer("pos_weight", pos_weight)
396         self.gamma_neg = gamma_neg
397         self.gamma_pos = gamma_pos
398         self.prob_margin = prob_margin
399
400     def forward(self, logits, labels):
401         mask = (labels >= 0).float()
402         labels_clamped = labels.clamp(min=0.0)
403         probs = torch.sigmoid(logits)
404
405         xs_pos = probs
406         loss_pos = F.binary_cross_entropy_with_logits(
407             logits, torch.ones_like(logits), pos_weight=self.pos_weight,
408             reduction='none'
409         )
410         focal_pos = (1.0 - xs_pos) ** self.gamma_pos
411
412         xs_neg = torch.clamp(probs - self.prob_margin, min=0.0)
413         loss_neg = -torch.log(torch.clamp(1.0 - xs_neg, min=1e-8))
414         focal_neg = xs_neg ** self.gamma_neg
415
416         loss = torch.where(
417             labels_clamped >= 0.5,
418             loss_pos * focal_pos,
419             loss_neg * focal_neg,
420         )
421         masked_loss = loss * mask
422         return masked_loss.sum() / mask.sum().clamp(min=1)

```

A.13 Negative Edge Subsampling

The following excerpt implements the batch-level negative edge subsampling described in Section 3.10, which rebalances each training batch to 30% positive edges.

```

460 def subsample_edges(edge_label, target_pos_rate=0.30):
461     pos_mask = (edge_label >= 0.5).any(dim=-1)
462     neg_mask = ~pos_mask
463     n_pos = pos_mask.sum().item()
464     if n_pos == 0:
465         return torch.arange(edge_label.shape[0])
466     n_neg_target = int(n_pos * (1 - target_pos_rate) / target_pos_rate)
467     neg_indices = neg_mask.nonzero(as_tuple=True)[0]
468     if len(neg_indices) <= n_neg_target:
469         return torch.arange(edge_label.shape[0])
470     perm = torch.randperm(len(neg_indices))[:n_neg_target]
471     pos_indices = pos_mask.nonzero(as_tuple=True)[0]
472     return torch.cat([pos_indices, neg_indices[perm]])

```

A.14 Inverse-Consistency Loss

The following excerpt implements the inverse-consistency loss term described in Section 3.10 and Equation 12, penalising disagreement between a relation and its predicted inverse.

```
485 def inverse_consistency_loss(logits, edge_index, inverse_pairs, rev_edge_map):
486     if rev_edge_map is None:
487         return torch.tensor(0.0, device=logits.device, requires_grad=True)
488     probs = torch.sigmoid(logits)
489     probs_rev = probs[rev_edge_map]
490     rel_indices = torch.tensor([p[0] for p in inverse_pairs], device=logits.device)
491     inv_indices = torch.tensor([p[1] for p in inverse_pairs], device=logits.device)
492     p_fwd = probs[:, rel_indices]
493     p_rev = probs_rev[:, inv_indices]
494     return ((p_fwd - p_rev) ** 2).mean() * 0.1
```

A.15 Data Augmentation: Gaussian Jitter

The following excerpt implements the Gaussian jitter transform and the full augmentation registry described in Section 3.10, one of the six geometric transforms used to expand the training set sevenfold.

```
271 def _jitter(x: torch.Tensor, edge_attr: torch.Tensor,
272             edge_label: torch.Tensor, sigma: float = 0.01) -> Tuple:
273     x = x.clone()
274     edge_attr = edge_attr.clone()
275
276     n_nodes = x.shape[0]
277     n_edges = edge_attr.shape[0]
278
279     node_noise = torch.randn(n_nodes, 3) * sigma
280     x[:, :3] = torch.clamp(x[:, :3] + node_noise, 0.0, 1.0)
281
282     edge_noise = torch.randn(n_edges, 3) * (sigma * 1.414)
283     edge_attr[:, :3] = torch.clamp(edge_attr[:, :3] + edge_noise, -1.0, 1.0)
284
285     return x, edge_attr, edge_label
286
287
288 AUGMENTATIONS = [
289     ("rotate_90", _rotate_z_90ccw),
290     ("rotate_180", _rotate_z_180),
291     ("rotate_270", _rotate_z_270ccw),
292     ("flip_x", _flip_x),
293     ("flip_y", _flip_y),
294     ("jitter", _jitter),
295     ("flip_z", _flip_z),
296 ]
```

A.16 Per-Relation Threshold Calibration

The following excerpt implements the per-relation decision threshold sweep described in Section 3.10, selecting the F1-maximising threshold for each relation on the validation set.

```
664 def tune_thresholds(model, val_loader, device):
665     model.eval()
666     all_logits, all_labels = [], []
667     with torch.no_grad():
668         for batch in val_loader:
669             batch = batch.to(device)
670             logits = model(batch.x, batch.edge_index, batch.edge_attr)
671             all_logits.append(logits.cpu())
672             all_labels.append(batch.y.cpu())
673     all_logits = torch.cat(all_logits, dim=0)
674     all_labels = torch.cat(all_labels, dim=0)
675
676     thresholds = {}
677     for rel_idx in range(NUM_RELATIONS):
678         best_f1, best_thresh = 0.0, 0.5
679         valid_mask = all_labels[:, rel_idx] >= 0
680         if valid_mask.sum() == 0:
681             thresholds[rel_idx] = 0.5
682             continue
683         probs = torch.sigmoid(all_logits[valid_mask, rel_idx]).numpy()
684         labels_valid = _binarize_labels(all_labels[valid_mask, rel_idx].numpy())
685         for thresh in np.arange(0.1, 0.9, 0.05):
686             preds = (probs >= thresh).astype(int)
687             f1 = f1_score(labels_valid, preds, zero_division=0)
688             if f1 > best_f1:
689                 best_f1 = f1
690                 best_thresh = float(thresh)
691         thresholds[rel_idx] = best_thresh
692     return thresholds
```

A.17 Symbolic Repair: Constraint Definition

The following excerpt defines the four logical constraint types, mutual exclusion, asymmetry, transitivity, and symmetry, described in Section 3.11 and Table 7.

```

41 MUTUAL_EXCLUSIONS = [
42     ("on_top_of", "under"),
43     ("higher_than", "lower_than"),
44     ("on_top_of", "lower_than"),
45     ("under", "higher_than"),
46     ("left_of", "right_of"),
47     ("in_front_of", "behind"),
48 ]
49
50 ASYMMETRIC_RELATIONS = {
51     "on_top_of", "under", "higher_than", "lower_than",
52     "left_of", "right_of", "in_front_of", "behind",
53 }
54
55 TRANSITIVE_RELATIONS = {
56     "higher_than", "lower_than", "left_of", "right_of",
57     "in_front_of", "behind",
58 }
59
60 SYMMETRIC_RELATIONS = {"adjacent_to"}

```

A.18 Symbolic Repair: Confidence Propagation

The following excerpt implements the inverse-completion confidence rule (Equation 17) and transitive-closure confidence rule (Equation 18) described in Section 3.11.

```

285 for (s, r, o), conf in working.items():
286     if r in INVERSE_PAIRS:
287         inv_key = (o, INVERSE_PAIRS[r], s)
288         if inv_key not in working and inv_key not in to_add:
289             to_add[inv_key] = conf * 0.95
290
291 for a, a_targets in adj.items():
292     for b, conf_ab in a_targets:
293         if b in adj:
294             for c, conf_bc in adj[b]:
295                 if c == a:
296                     continue
297                 trans_key = (a, r, c)
298                 if trans_key not in working:
299                     implied_conf = min(conf_ab, conf_bc) * 0.9

```

A.19 Recall@k Computation

The following excerpt implements the Recall@k metric described in Section 3.12 and Equation 14, used throughout the evaluation in Chapter 4.

```
739 def compute_recall_at_k(probs, labels, k):
740     hits, total = 0, 0
741     for edge_idx in range(probs.shape[0]):
742         edge_labels = labels[edge_idx]
743         edge_probs = probs[edge_idx]
744         positive_rels = np.where(edge_labels >= 0.5)[0]
745         if len(positive_rels) == 0:
746             continue
747         top_k_indices = np.argsort(edge_probs)[::-1][:k]
748         for rel_idx in positive_rels:
749             total += 1
750             if rel_idx in top_k_indices:
751                 hits += 1
752     return hits / max(total, 1)
```

A.20 Warmup + Cosine Learning Rate Schedule

The following excerpt implements the linear-warmup, cosine-decay learning rate schedule described in Section 3.10 (Optimisation and Schedule).

```
761 class WarmupCosineScheduler:
762     def __init__(self, optimizer, warmup_epochs, total_epochs):
763         self.optimizer = optimizer
764         self.warmup_epochs = warmup_epochs
765         self.total_epochs = total_epochs
766         self.base_lrs = [pg['lr'] for pg in optimizer.param_groups]
767
768     def step(self, epoch):
769         if epoch < self.warmup_epochs:
770             scale = (epoch + 1) / self.warmup_epochs
771         else:
772             progress = (epoch - self.warmup_epochs) / max(1, self.total_epochs - self.warmup_epochs)
773             scale = 0.5 * (1.0 + np.cos(np.pi * progress))
774         for pg, base_lr in zip(self.optimizer.param_groups, self.base_lrs):
775             pg['lr'] = base_lr * scale
```

Proforma 4

Undertaking from the PG student while submitting his/her final dissertation to his respective institute

Ref. No. _____

I , the following student

Sr. No.	Sequence of students names on a dissertation	Students name	Name of the Institute & Place	Email & Mobile
1.	First Author	Debjyoti Sengupta	SIG	Email: debjyotiashu66@gmail.com / Mobile: 7718395692

hereby give an undertaking that the dissertation 'LogicSplat: Neuro-Symbolic 3D Scene Graph Generation via Geometric Kolmogorov–Arnold Networks' has been checked for its Similarity Index/Plagiarism through Turnitin software tool; and that the document has been prepared by me and it is my original work and free of any plagiarism. It was found that:

1.	The Similarity Index (SI) was: <i>(Note: SI range: 0 to 10%; if SI is > 10%, then authors cannot communicate ms; attachment of SI report is mandatory)</i>	7%
2.	The ethical clearance for research work conducted obtained from: <i>(Note: Name the consent obtaining body; if 'not applicable' then write so)</i>	NA
3.	The source of funding for research was: <i>(Note: Name the funding agency; or write 'self' if no funding source is involved)</i>	Self
4.	Conflict of interest: <i>(Note: Tick ✓ whichever is applicable)</i>	No
5.	The material (adopted text, tables, figures, graphs, etc.) as has been obtained from other sources, has been duly acknowledged in the manuscript: <i>(Note: Tick ✓ whichever is applicable)</i>	Yes

In case if any of the above-furnished information is found false at any point in time, then the University authorities can take action as deemed fit against all of us.

Full Name & Signature of the student

Name & Signature of SIU Guide/Mentor

Date: 10-07-2026

Endorsement by

Academic Integrity Committee (AIC)

Place: Pune

Note: It is mandatory that the Similarity Index report of plagiarism (only first page) should be appended to the UG/PG dissertation

Debjoyti Sengupta

On ☒

7%

1	Submitted to Symbiosi... Student Paper	2%	>
2	arxiv.org Internet Source	1%	>
3	www.cjig.cn Internet Source	<1%	>
4	Xingxing Zuo, Pouya Sa... Publication	<1%	>
5	theses.gla.ac.uk Internet Source	<1%	>
6	nslab-cuk.github.io Internet Source	<1%	>
7	pure.eur.nl Internet Source	<1%	>
8	repositum.tuwien.at Internet Source	<1%	>
9	portal.mvmtum.de	<1%	>